COMPUTER LABORATORY MANUAL

IQRA UNIVERSITY IU

Islamabad Campus

CMC381-L

Artificial Intelligence Lab

BS (Computer Science) | Credit Hours: 3+1 | Fall 2026

Student Name _____

Registration No. _____

Section _____

Prepared by: Faiza Ali, Junior Lecturer

DEPARTMENT OF COMPUTER SCIENCE

Faculty of Engineering Sciences and Technology (FEST)

Iqra University, Islamabad

PREFACE

This laboratory manual has been prepared to help undergraduate students of Computer Science study, understand and implement the core algorithms of Artificial Intelligence. It follows the HEC National Curriculum for the BS Computer Science core course *Artificial Intelligence* and the course outline of **CMC381-L Artificial Intelligence Lab** at Iqra University. The prerequisite for this course is **CMC371**, and students are expected to have basic programming skills.

The manual uses **Python** as the programming language because of its readability and rich ecosystem of AI libraries (NumPy, scikit-learn, Matplotlib, NetworkX, PyTorch/TensorFlow). The twelve experiments move from intelligent agents and problem formulation, through uninformed, informed, local and adversarial search, to fuzzy logic and machine learning, ending with artificial neural networks. Every experiment contains the theory needed for the session, a worked and tested example program with its output, graded lab tasks, a post-lab (home) task, and viva questions.

Students' understanding will be evaluated through lab tasks graded on rubrics, a midterm lab assessment, an **Open-Ended Lab (OEL) / project** carried out in groups, and a final lab examination.

PREPARED BY

This lab manual has been prepared by **Faiza Ali**, Junior Lecturer, Department of Computer Science, Iqra University Islamabad (faiza.ali@iqraisb.edu.pk), for the Fall 2026 semester.

GENERAL INSTRUCTIONS

1. Students must bring this manual (printed or digital) to every lab session and keep it until the end of the semester.
2. Read the theory and the example code of an experiment **before** coming to the lab session.
3. Each lab task must be completed and demonstrated to the instructor in the same session unless stated otherwise.
4. Save your work as `LabXX_RegNo.ipynb` (or `.py`) and upload it to the LMS before the deadline announced by the instructor.
5. Write clean, indented and commented code. Use meaningful variable and function names.
6. Plagiarism is strictly forbidden. Copied work (from peers or AI tools) receives zero marks for all rubrics of that lab and no resubmission is entertained.
7. Marks will be deducted for late submissions as per departmental policy.
8. Mobile phones must be on silent mode. Eating and drinking are not allowed in the computer lab.
9. Shut down your system properly and leave the workstation clean at the end of the session.

VERSION HISTORY

Date	Updated By	Details
October 2026	Faiza Ali (Junior Lecturer)	First version: 12 experiments aligned with the CMC381-L course outline and weekly plan, lab rubrics, CLO mapping and Open-Ended Lab.

COURSE INFORMATION

1. Basic Information

Course Code	CMC381-L	**Course Name**	Artificial Intelligence Lab
Credit Hours	3+1	**Prerequisite(s)**	CMC371
Instructor	Faiza Ali	**Designation**	Junior Lecturer
Email	faiza.ali@iqraisb.edu.pk	**Semester**	Fall 2026
Consulting Hours	As announced on LMS	**Lab Duration**	3 contact hours / week

2. Course Objective

Artificial Intelligence has emerged as one of the most significant and promising areas of computing. This course focuses on the foundations of AI and its basic techniques such as symbolic manipulation, pattern matching, knowledge representation, decision making, and appreciating the differences between knowledge, data and code. Python is used for the practical work of this course.

3. Course Contents

Introduction to AI, intelligence and artificial intelligence, branches and applications. Agent-based systems: introduction, applications, rationality, environment types and agent types. Problem solving and problem formulation. Uninformed search strategies: breadth-first search, uniform cost search, depth-first search, iterative deepening search; performance parameters. Informed (heuristic) search strategies: heuristic functions, greedy search, A* search. Genetic algorithm. Game playing: minimax algorithm, alpha-beta pruning. Knowledge and reasoning, introduction to fuzzy logic: operators and inference procedure. Advanced topics: machine learning, types of machine learning, artificial neural networks, Naive Bayes. Natural language processing. Recent trends and applications of AI algorithms.

4. Curriculum Alignment

The experiments are aligned with the HEC National Curriculum for BS Computer Science (core course: Artificial Intelligence) and with the **Intelligent Systems (IS)** knowledge area of the ACM/IEEE-CS Computer Science Curricula. The course also supports **UN Sustainable Development Goal 9** (Industry, Innovation and Infrastructure).

COURSE LEARNING OUTCOMES (CLOs)

At the end of the course, students will be able to:

CLO	CLO Statement	BT Level	GA	ACM KA	SDG	% Weight
CLO1	Apply fundamental concepts of Artificial Intelligence, including rational agents, search algorithms, and fuzzy logic, to solve real-world problems.	C3	GA2	IS	9	60%
CLO2	Leverage modern AI tools and frameworks to design and implement basic algorithms, including search strategies and machine learning models.	P2	GA5			40%

Note: On successful completion of the course, GA1 (Academic Education) will automatically attain. BT = Bloom's Taxonomy level (C = Cognitive, P = Psychomotor), GA = Graduate Attribute, KA = Knowledge Area, SDG = Sustainable Development Goal.

Assessment Scheme (Lab)

Assessment	Description	Weight
Lab Tasks	In-lab tasks of Labs 1–12, graded on Rubrics Group 1	25%
Midterm Lab Assessment	Practical exam on Labs 1–6 (Week 8)	15%
Open-Ended Lab / Project	Group work assigned in Week 7, assessed in Weeks 15–16 (Rubrics Group 2)	20%
Final Lab Examination	Practical exam + viva on all labs (Week 17)	40%

The weights above are suggested; the instructor may adjust them according to the departmental assessment policy announced at the start of the semester.

LAB RUBRICS

Every lab task is assessed on five criteria (R1–R5), each marked from 0 to 3, giving a total of 15 marks per lab. **Group 1** is used for individual lab experiments; **Group 2** is used for the Open-Ended Lab / Project.

Lab Rubrics (Group 1) – Individual Lab Experiments

Criteria	Unacceptable (Marks = 0)	Substandard (Marks = 1)	Adequate (Marks = 2)	Proficient (Marks = 3)
R1 Completeness and Accuracy	The program failed to produce the right, accurate result.	The program produced inaccurate or incomplete results; not all features were implemented.	The program was correctly functional and most of the features were implemented.	The program was correctly functional and all the features were implemented.
R2 Syntax and Semantics	The student fails to find the syntax and semantic errors in the program.	The student finds a few syntax and semantic errors with extensive guidance.	The student finds most of the syntax and semantic errors with minimum guidance.	The student finds all syntax and semantic errors without any guidance.
R3 Demonstration	The student failed to demonstrate a clear understanding of the assigned task.	The student has basic understanding, but asked questions were not answered.	The student has basic understanding and gives fundamental answers to the asked questions.	The student demonstrated accurate understanding of the lab objective and concepts; all questions answered completely and correctly.
R4 Complexity and Readability	The code is poorly organised and very difficult to read.	The code is readable only by someone who knows what it is supposed to do.	The code is fairly easy to read.	The code is exceptionally well organised, commented and easy to follow.
R5 Perseverance and Plagiarism	The complete program is copied, showing no effort by the student; results in zero for all rubrics.	Most of the working program is copied; minor contribution by the student.	Most of the working program is contributed by the student; minor copied components.	The complete working program is contributed by the student.

Lab Rubrics (Group 2) – Open-Ended Lab / Project

Criteria	Unacceptable (Marks = 0)	Substandard (Marks = 1)	Adequate (Marks = 2)	Proficient (Marks = 3)
R1 Problem Formulation and Design	No clear formulation of states, actions, goal or performance measure.	Partial formulation; important elements missing or incorrect.	Correct formulation with minor gaps; design is reasonable.	Complete, well-justified formulation and design; alternatives considered.
R2 Implementation and Results	The system failed to produce the right result.	The system produced inaccurate or incomplete results.	The system was correctly functional and most features were implemented.	The system was correctly functional, all features were implemented and results were analysed.
R3 Demonstration and Viva	Failed to demonstrate a clear understanding of the task.	Basic understanding, but questions were not answered.	Moderate understanding; answers to questions are basic.	Accurate understanding; all questions answered completely and correctly.
R4 Contribution / Group Participation	Shows little commitment to group goals and fails to perform assigned roles.	Demonstrates commitment but has difficulty performing assigned roles.	Demonstrates commitment and carries out assigned roles effectively.	Actively helps to identify group goals and works effectively in all roles.
R5 Report and Presentation	Poor report and presentation; cannot explain the topic; terminology lacking.	Report/presentation lacks clarity and organisation; little use of technical terms.	Acceptable report and presentation; adequate use of technical terms.	Well-organised, clear report and presentation; good use of technical vocabulary.

MAPPING OF LAB EXPERIMENTS

Week	Lab No	Experiment	CLO	Rubric Group
1	1	Introduction to AI, Intelligent Agents and Environments	CLO1	G1
2	2	Problem Formulation and the 8-Puzzle Problem	CLO1	G1
3	3	Uninformed Search: BFS, DFS and Uniform Cost Search	CLO1, CLO2	G1
4	4	Depth-Limited and Iterative Deepening Search	CLO1, CLO2	G1
5	5	Informed Search: Greedy Best-First and A* Search	CLO1, CLO2	G1
6	6	Local Search and Optimization: Hill Climbing and Genetic Algorithm	CLO1, CLO2	G1
7	–	Open-Ended Lab / Project Assigned	CLO1, CLO2	G2
8	–	Midterm Exam	–	–
9	7	Adversarial Search: Minimax and Alpha-Beta Pruning	CLO1, CLO2	G1
10	8	Fuzzy Sets and Fuzzy Set Operations	CLO1	G1
11	9	Fuzzy Logic Based Expert System (AC Temperature Control)	CLO1, CLO2	G1
12	10	Supervised and Unsupervised Learning: Naive Bayes and K-Means	CLO2	G1
13	11	Artificial Neural Networks: The Perceptron	CLO2	G1
14	12	Multi-Layer Perceptron and Backpropagation	CLO2	G1
15	–	Revision / Open-Ended Lab / Project Assessment	CLO1, CLO2	G2
16	–	Open-Ended Lab / Project Assessment	CLO1, CLO2	G2
17	–	Final Exam	CLO1, CLO2	–

MARKS SHEET

Date	Experiment	Max Marks	R1	R2	R3	R4	R5	Instructor Sign
	Lab 1: Intelligent Agents	15						
	Lab 2: Problem Formulation: 8-Puzzle	15						
	Lab 3: BFS, DFS and UCS	15						
	Lab 4: DLS and IDDFS	15						
	Lab 5: Greedy Best-First and A*	15						
	Lab 6: Hill Climbing and GA	15						
	Lab 7: Minimax and Alpha-Beta	15						
	Lab 8: Fuzzy Sets and Operations	15						
	Lab 9: Fuzzy Expert System	15						
	Lab 10: Naive Bayes and K-Means	15						
	Lab 11: Perceptron	15						
	Lab 12: MLP and Backpropagation	15						
	Open-Ended Lab / Project	15						
Grand Total		**195**						

LIST OF EXPERIMENTS

Lab Environment Setup..... Week 1
Lab # 1: Introduction to AI, Intelligent Agents and Environments.....Week 1
Lab # 2: Problem Formulation and the 8-Puzzle Problem.....Week 2
Lab # 3: Uninformed Search: BFS, DFS and Uniform Cost Search.....Week 3
Lab # 4: Depth-Limited and Iterative Deepening Search.....Week 4
Lab # 5: Informed Search: Greedy Best-First and A* Search.....Week 5
Lab # 6: Local Search and Optimization: Hill Climbing and Genetic Algorithm.....Week 6
Open-Ended Lab / Project: Smart Campus Delivery Robot (assigned).....Week 7
Lab # 7: Adversarial Search: Minimax and Alpha-Beta Pruning.....Week 9
Lab # 8: Fuzzy Sets and Fuzzy Set Operations.....Week 10
Lab # 9: Fuzzy Logic Based Expert System (AC Temperature Control).....Week 11
Lab # 10: Supervised and Unsupervised Learning: Naive Bayes and K-Means.....Week 12
Lab # 11: Artificial Neural Networks: The Perceptron.....Week 13
Lab # 12: Multi-Layer Perceptron and Backpropagation.....Week 14
Appendix A: Lab Exam Guidelines.....Weeks 8 & 17
Appendix B: Lab Report Template..... –

LAB ENVIRONMENT SETUP

All experiments in this manual are written for **Python 3.10 or later**. You may work on the lab computers (Anaconda installed) or on your own laptop. Google Colab can be used when a local installation is not possible.

Option A: Anaconda (recommended for lab computers)

1. Download and install Anaconda Individual Edition from https://www.anaconda.com/download.
2. Open **Anaconda Prompt** (Windows) or a terminal (Linux/macOS).
3. Create an environment for this course and activate it:

Commands

```
conda create -n ailab python=3.11 -y
conda activate ailab
pip install numpy matplotlib networkx scikit-learn pandas jupyter
pip install torch torchvision          # needed only for Lab 12
jupyter notebook
```

Option B: Google Colab

Open https://colab.research.google.com, sign in with your Google account and create a new notebook. NumPy, Matplotlib, NetworkX, scikit-learn, PyTorch and TensorFlow are pre-installed. Use **Runtime → Change runtime type → GPU** for Lab 12 if training is slow.

Verify your installation

Example Code

```
import sys, numpy, sklearn, matplotlib, networkx
print("Python      :", sys.version.split()[0])
print("NumPy       :", numpy.__version__)
print("scikit-learn:", sklearn.__version__)
print("All libraries imported successfully!")
```

Libraries used in this manual

Library	Purpose	Used in
collections, heapq, itertools	Queues, stacks and priority queues for search	Labs 2–5
random, math	Random states, genetic operators, game utilities	Labs 1, 6, 7
NumPy	Numerical arrays, fuzzy membership functions, neural networks	Labs 8, 9, 11, 12
Matplotlib, NetworkX	Plotting graphs, search trees and results	Labs 4, 8, 10
scikit-learn	Naive Bayes, K-Means, datasets and metrics	Lab 10
PyTorch (or TensorFlow/Keras)	Multi-layer perceptron on MNIST	Lab 12

Lab # 1: Introduction to AI, Intelligent Agents and Environments

Objective: To understand the scope of AI, the types of intelligent agents and task environments, and to implement a simple reflex agent in Python.

Week / Lab No.	:	Week 1 / Lab 1
CLO Mapping	:	CLO1
Time Required	:	3 hrs
Programming Language	:	Python 3.x
Software Required	:	Anaconda (Jupyter Notebook) / Google Colab
Libraries	:	random (standard library)

Learning Outcomes

- Describe what AI is and identify real-world AI applications.
- Specify a task environment using the **PEAS** framework and classify its properties.
- Distinguish between the five types of agents.
- Implement and evaluate a simple reflex agent for the two-square vacuum world.

Theory

1.1 What is Artificial Intelligence?

Artificial Intelligence (AI) is the study and design of **intelligent agents**: systems that perceive their environment and take actions that maximise their chance of achieving their goals. Russell and Norvig group definitions of AI into four approaches: thinking humanly (cognitive modelling), acting humanly (Turing test), thinking rationally (laws of thought) and **acting rationally (the rational agent approach)**, which is the approach followed in this lab.

1.2 Applications of AI

Application Area	Example	AI Technique
Computer vision	Face unlock, Google Lens, medical image diagnosis	Convolutional neural networks
Natural language processing	Chatbots (ChatGPT, Claude), machine translation, Urdu speech-to-text	Large language models, transformers
Robotics	Robot vacuum cleaners, warehouse robots, self-driving cars	Agents, planning, search, sensors
Games	Chess engines, AlphaGo	Adversarial search, reinforcement learning
Recommendation	YouTube, Daraz, Netflix suggestions	Machine learning, collaborative filtering
Agriculture and health	Crop disease detection, diabetic retinopathy screening	Image classification

1.3 Agents and Environments

An **agent** is anything that perceives its **environment** through **sensors** and acts upon that environment through **actuators**. The agent's behaviour is described by the **agent function**, which maps any sequence of percepts to an action. The **agent program** is the concrete implementation of the agent function running on a physical architecture.

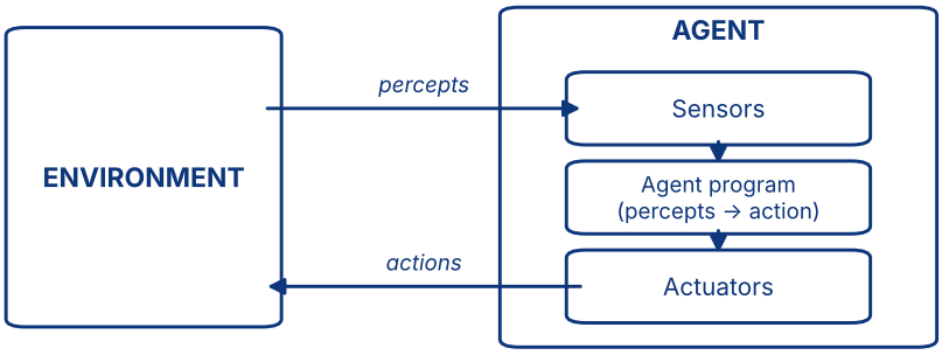


Figure 1.1: An agent interacts with its environment through sensors and actuators

1.4 Rationality and PEAS

A **rational agent** selects, for every possible percept sequence, the action that is expected to maximise its **performance measure**, given the evidence in the percept sequence and its built-in knowledge. To design an agent, we first specify the task environment using **PEAS**: Performance measure, Environment, Actuators and Sensors.

Agent	Performance Measure	Environment	Actuators	Sensors
Robot vacuum	Cleanliness, battery use, time	Rooms, dirt, furniture	Wheels, suction brush	Camera, dirt sensor, bump sensor
Automated taxi	Safety, speed, legality, comfort	Roads, traffic, pedestrians	Steering, brake, accelerator, horn	Cameras, GPS, speedometer, LIDAR
Medical diagnosis system	Healthy patient, low cost	Patient, hospital, staff	Display of tests, diagnoses	Keyboard entry of symptoms, reports

1.5 Properties of Task Environments

- **Fully vs partially observable:** can the sensors see the complete state of the environment?
- **Deterministic vs stochastic:** is the next state completely determined by the current state and action?
- **Episodic vs sequential:** does the current decision affect future decisions?
- **Static vs dynamic:** can the environment change while the agent is deliberating?
- **Discrete vs continuous:** are states, time, percepts and actions discrete?
- **Single-agent vs multi-agent:** are there other agents whose behaviour affects this agent?

1.6 Types of Agents

1. **Simple reflex agent:** selects actions using only the current percept, through condition–action (if–then) rules.
2. **Model-based reflex agent:** keeps an internal state that tracks the parts of the world it cannot currently see.
3. **Goal-based agent:** chooses actions that lead towards an explicit goal (uses search and planning).
4. **Utility-based agent:** chooses actions that maximise a utility function (degree of "happiness").
5. **Learning agent:** improves its performance over time using feedback from a critic.

1.7 The Vacuum-Cleaner World

The vacuum world has two locations, **A** and **B**. Each location is either Clean or Dirty, and the agent is in one of them. The agent perceives `[location, status]` and can perform the actions **Left**, **Right** and **Suck**. A simple reflex agent for this world is shown below.

Pseudocode

```
function REFLEX-VACUUM-AGENT([location, status]) returns an action
    if status = Dirty then return Suck
    else if location = A then return Right
    else if location = B then return Left
```

Implementation

The program below models the environment as a class, implements the agent as a function, and measures performance: +10 for every square cleaned and −1 for every move.

Example Code

```
import random

class VacuumEnvironment:
    """Two-square world: locations 'A' and 'B', each Clean (0) or Dirty (1)."""
    def __init__(self):
        self.status = {'A': random.randint(0, 1), 'B': random.randint(0, 1)}
        self.agent_location = random.choice(['A', 'B'])

    def percept(self):
        return (self.agent_location, self.status[self.agent_location])

    def execute(self, action):
        if action == 'Suck':
            self.status[self.agent_location] = 0
        elif action == 'Right':
            self.agent_location = 'B'
        elif action == 'Left':
            self.agent_location = 'A'

def reflex_vacuum_agent(percept):
    """Condition-action rules: decide using ONLY the current percept."""
    location, dirty = percept
    if dirty:
        return 'Suck'
    elif location == 'A':
        return 'Right'
    else:
        return 'Left'

random.seed(7)
env = VacuumEnvironment()
env.status = {'A': 1, 'B': 1}; env.agent_location = 'B'
print("Initial state :", env.status, "| Agent at", env.agent_location)
score = 0
for step in range(1, 5):
    p = env.percept()
    action = reflex_vacuum_agent(p)
    env.execute(action)
    if action == 'Suck':
        score += 10      # reward for cleaning
    else:
        score -= 1       # cost of moving
    print(f"Step {step}: percept={p} -> action={action:5s} | world={env.status}")
print("Final state   :", env.status)
print("Performance score:", score)
```

Output

```
Initial state : {'A': 1, 'B': 1} | Agent at B
Step 1: percept=('B', 1) -> action=Suck  | world={'A': 1, 'B': 0}
Step 2: percept=('B', 0) -> action=Left  | world={'A': 1, 'B': 0}
Step 3: percept=('A', 1) -> action=Suck  | world={'A': 0, 'B': 0}
Step 4: percept=('A', 0) -> action=Right | world={'A': 0, 'B': 0}
Final state   : {'A': 0, 'B': 0}
Performance score: 18
```

Observation: In Step 4 both squares are already clean, yet the agent still moves Right and loses a point. A simple reflex agent has **no memory**: it reacts only to the current percept, so it cannot know that its job is finished.

Lab Tasks

1. **AI applications:** Explore any two AI tools (e.g., Google Lens for image recognition and a chatbot such as ChatGPT or Claude). For each, write the input, the output, the AI technique you think is used, and one limitation you observed.
2. **PEAS:** Write the PEAS description and classify the environment properties (six properties of Section 1.5) for: (a) a chess-playing program with a clock, (b) an online English tutor, (c) a drone delivering parcels across Islamabad.
3. **Random agent:** Modify the program so that the agent selects a random action from {Left, Right, Suck}. Run both agents for 20 steps on 10 random initial states and compare their average performance scores.
4. **Model-based agent:** Extend the reflex agent so that it remembers which locations it has already cleaned and performs **NoOp** (stops) when both locations are clean. Show that its performance score is higher than the simple reflex agent when moves are penalised.

Post-Lab Task

Extend the vacuum world to a **3 × 3 grid** of rooms with random dirt. Implement a model-based reflex agent that visits every room and cleans it. Report the number of moves, the number of cleaned rooms and the performance score.

Viva Questions

1. What is the difference between an agent function and an agent program?
2. Why is a simple reflex agent not suitable for a partially observable environment?
3. Is the vacuum world in this lab deterministic or stochastic? Justify.
4. Give an example of a utility-based agent from daily life.

References

1. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2021, Chapters 1–2.
2. Python documentation: https://docs.python.org/3/tutorial/

Summary

This lab introduced AI, rational agents, PEAS and environment properties, and gave hands-on experience of implementing and evaluating a simple reflex agent in Python.

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab # 2: Problem Formulation and the 8-Puzzle Problem

Objective: To learn how to formulate a problem as a search problem, implement the 8-puzzle problem, and measure solving performance by tracking node expansion.

Week / Lab No.	:	Week 2 / Lab 2
CLO Mapping	:	CLO1
Time Required	:	3 hrs
Programming Language	:	Python 3.x
Software Required	:	Anaconda (Jupyter Notebook) / Google Colab
Libraries	:	collections, time

Learning Outcomes

- Formulate a problem in terms of initial state, actions, transition model, goal test and path cost.
- Represent the 8-puzzle state space in Python.
- Check the solvability of an 8-puzzle configuration.
- Measure search performance using nodes expanded, nodes generated and time.

Theory

2.1 Problem-Solving Agents

A **problem-solving agent** is a goal-based agent that plans ahead: it considers a sequence of actions that forms a path to a goal state. This process is called **search**. Before searching, the problem must be formulated precisely.

2.2 Components of a Well-Defined Problem

1. **State space:** the set of all states reachable from the initial state.
2. **Initial state:** the state in which the agent starts.
3. **Actions:** `ACTIONS(s)` returns the legal actions in state *s*.
4. **Transition model:** `RESULT(s, a)` returns the state reached by doing action *a* in state *s*.
5. **Goal test:** determines whether a given state is a goal state.
6. **Path cost:** a function that assigns a numeric cost to each path; the **optimal solution** has the lowest path cost.

2.3 The 8-Puzzle Problem

The 8-puzzle consists of a 3 × 3 board with eight numbered tiles and one blank space. A tile adjacent to the blank can slide into it. The objective is to reach the goal configuration.

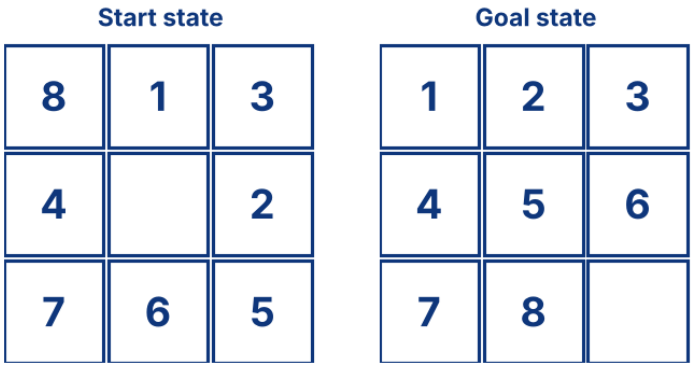


Figure 2.1: An example start state and the goal state of the 8-puzzle

Component	8-Puzzle Formulation
States	Any arrangement of tiles 1–8 and the blank on the 3 × 3 board (9!/2 = 181,440 reachable states)
Initial state	Any given configuration, e.g., (1, 2, 3, 4, 0, 6, 7, 5, 8) where 0 = blank
Actions	Move the blank Up, Down, Left or Right (when not blocked by the edge)
Transition model	Swap the blank with the neighbouring tile in the chosen direction
Goal test	State equals (1, 2, 3, 4, 5, 6, 7, 8, 0)
Path cost	Each move costs 1; path cost = number of moves

2.4 Solvability

Only half of all tile arrangements can reach the goal. For a 3 × 3 board, count the number of **inversions** (pairs of tiles *a*, *b* where *a* appears before *b* but *a* > *b*, ignoring the blank). The puzzle is solvable if and only if the number of inversions is **even**.

2.5 Measuring Search Performance

- **Completeness:** is the algorithm guaranteed to find a solution when one exists?
- **Optimality:** does it find the lowest-cost solution?
- **Time complexity:** measured by the number of nodes **expanded** (or generated).
- **Space complexity:** maximum number of nodes stored in memory.

Implementation

The program formulates the 8-puzzle, checks solvability, solves it with breadth-first search and reports performance counters. (Execution time will differ on your machine.)

Example Code

```
from collections import deque
import time

GOAL = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)          # 0 represents the blank tile

MOVES = {'Up': -3, 'Down': 3, 'Left': -1, 'Right': 1}

def actions(state):
    """Return the legal moves of the blank tile."""
    i = state.index(0)
    row, col = divmod(i, 3)
    legal = []
    if row > 0: legal.append('Up')
    if row < 2: legal.append('Down')
    if col > 0: legal.append('Left')
    if col < 2: legal.append('Right')
    return legal

def result(state, action):
    """Transition model: swap the blank with the neighbouring tile."""
    i = state.index(0)
    j = i + MOVES[action]
    s = list(state)
    s[i], s[j] = s[j], s[i]
    return tuple(s)
```

```
def goal_test(state):
    return state == GOAL

def is_solvable(state):
    tiles = [t for t in state if t != 0]
    inversions = sum(1 for a in range(len(tiles))
                     for b in range(a + 1, len(tiles)) if tiles[a] > tiles[b])
    return inversions % 2 == 0

def solve_bfs(start):
    frontier = deque([(start, [])])
    explored = {start}
    expanded = 0
    while frontier:
        state, path = frontier.popleft()
        if goal_test(state):
            return path, expanded, len(explored)
        expanded += 1
        for a in actions(state):
            child = result(state, a)
            if child not in explored:
                explored.add(child)
                frontier.append((child, path + [a]))
    return None, expanded, len(explored)

def show(state):
    for r in range(3):
        print(' '.join(str(x) if x else '_' for x in state[r*3:r*3+3]))

start = (1, 2, 3,
         4, 0, 6,
         7, 5, 8)
print("Start state:"); show(start)
print("Solvable:", is_solvable(start))
t0 = time.perf_counter()
path, expanded, generated = solve_bfs(start)
t1 = time.perf_counter()
print("Solution :", path)
print("Moves    :", len(path))
print("Nodes expanded :", expanded)
print("Nodes generated:", generated)
print(f"Time taken     : {(t1-t0)*1000:.2f} ms")
```

Output

```
Start state:
1 2 3
4 _ 6
7 5 8
Solvable: True
Solution : ['Down', 'Right']
Moves    : 2
Nodes expanded : 8
Nodes generated: 16
Time taken     : 0.07 ms
```

Lab Tasks

1. Run the program for the start state (8, 1, 3, 4, 0, 2, 7, 6, 5). Record the solution length, nodes expanded, nodes generated and time.
2. Write a function `print_solution(start, path)` that prints the board after every move of the solution.
3. Generate 5 random **solvable** start states by applying 5, 10, 15, 20 and 25 random moves to the goal. Solve each and tabulate depth vs nodes expanded. Plot the result using Matplotlib and comment on the growth.
4. Test your program with the unsolvable state (1, 2, 3, 4, 5, 6, 8, 7, 0). What does your program print? Make sure it never starts searching an unsolvable state.

Post-Lab Task

Formulate the **Missionaries and Cannibals** problem (3 missionaries, 3 cannibals, one boat for two). Write the state representation, actions, transition model and goal test, and solve it in Python using the same framework.

Viva Questions

1. What is the difference between a state and a node?
2. Why do we keep an explored set during search?
3. How many states are reachable in the 8-puzzle, and why only half of 9!?
4. What is the branching factor of the 8-puzzle?

References

1. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Chapter 3.
2. https://en.wikipedia.org/wiki/15_puzzle (solvability)

Summary

This lab covered problem formulation and the 8-puzzle. Students implemented the state space and measured search performance through node-expansion counts, which will be used to compare algorithms in the following labs.

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab # 3: Uninformed Search: BFS, DFS and Uniform Cost Search

Objective: To understand and implement Breadth-First Search, Depth-First Search and Uniform Cost Search, and compare their performance on a graph-based maze and a weighted road map.

Week / Lab No.	:	Week 3 / Lab 3
CLO Mapping	:	CLO1, CLO2
Time Required	:	3 hrs
Programming Language	:	Python 3.x
Software Required	:	Anaconda (Jupyter Notebook) / Google Colab
Libraries	:	collections, heapq

Learning Outcomes

- Implement BFS with a FIFO queue, DFS with a LIFO stack and UCS with a priority queue.
- Represent mazes and graphs in Python using adjacency lists and grids.
- Compare uninformed strategies in terms of completeness, optimality, time and space.

Theory

3.1 Uninformed (Blind) Search

Uninformed search strategies use only the information available in the problem definition. They generate successors and distinguish a goal state from a non-goal state, but have no knowledge of how close a state is to the goal. The strategies differ only in the **order in which nodes are expanded**, which is controlled by the data structure used for the frontier.

3.2 Breadth-First Search (BFS)

BFS expands the **shallowest** unexpanded node first using a **FIFO queue**. It is complete and optimal when all step costs are equal, but its time and space complexity are O(b^d), where *b* is the branching factor and *d* the depth of the shallowest solution.

3.3 Depth-First Search (DFS)

DFS expands the **deepest** unexpanded node first using a **LIFO stack** (or recursion). It needs only O(bm) memory, where *m* is the maximum depth, but it is not optimal and may not be complete in infinite state spaces.

3.4 Uniform Cost Search (UCS)

UCS expands the node with the **lowest path cost g(n)** using a **priority queue**. It is complete and optimal for any positive step costs. When all step costs are equal, UCS behaves like BFS.

Pseudocode

```
function UNIFORM-COST-SEARCH(problem) returns a solution or failure
    frontier <- priority queue ordered by g, containing the initial node
    explored <- empty set
    loop
        if frontier is empty then return failure
        node <- POP(frontier)                  // lowest g(n)
        if GOAL-TEST(node.state) then return SOLUTION(node)
        add node.state to explored
        for each action in ACTIONS(node.state) do
            child <- CHILD-NODE(problem, node, action)
            if child.state not in explored then insert child into frontier
```

Criterion	BFS	DFS	UCS
Complete?	Yes (finite b)	No (infinite depth)	Yes (positive costs)
Optimal?	Yes (equal costs)	No	Yes
Time	O(b^d)	O(b^m)	O(b^(1+C*/ε))
Space	O(b^d)	O(bm)	O(b^(1+C*/ε))
Frontier	FIFO queue	LIFO stack	Priority queue on g(n)

3.5 The Maze

The maze below is a 5 × 7 grid. **S** is the start, **G** is the goal and black cells are walls. The agent can move Up, Right, Down or Left. Each free cell is a node of a graph and neighbouring free cells are connected by edges.

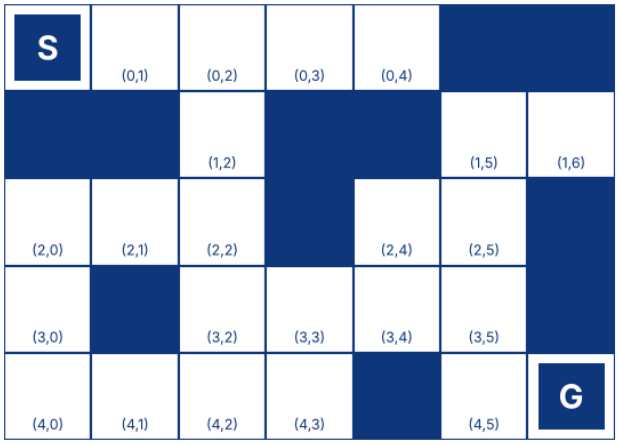


Figure 3.1: Graph-based maze used in this lab

Implementation

Example Code

```
from collections import deque
import heapq

# ---------- Grid maze: S = start, G = goal, # = wall ----------
MAZE = ["S . . . . # #",
        "# # . # # . .",
        ". . . # . . #",
        ". # . . . . #",
        ". . . . # . G"]
grid = [row.split() for row in MAZE]
ROWS, COLS = len(grid), len(grid[0])

def find(ch):
    for r in range(ROWS):
        for c in range(COLS):
            if grid[r][c] == ch:
                return (r, c)

def neighbours(cell):
    r, c = cell
    for dr, dc in [(-1, 0), (0, 1), (1, 0), (0, -1)]:   # Up, Right, Down, Left
        nr, nc = r + dr, c + dc
        if 0 <= nr < ROWS and 0 <= nc < COLS and grid[nr][nc] != '#':
```

```
            yield (nr, nc)

def bfs(start, goal):
    frontier = deque([start]); parent = {start: None}; expanded = 0
    while frontier:
        node = frontier.popleft()                # FIFO queue
        expanded += 1
        if node == goal:
            return build_path(parent, goal), expanded
        for nb in neighbours(node):
            if nb not in parent:
                parent[nb] = node
                frontier.append(nb)
    return None, expanded

def dfs(start, goal):
    frontier = [start]; parent = {start: None}; expanded = 0
    visited = set()
    while frontier:
        node = frontier.pop()                    # LIFO stack
        if node in visited:
            continue
        visited.add(node); expanded += 1
        if node == goal:
            return build_path(parent, goal), expanded
        for nb in neighbours(node):
            if nb not in visited:
                parent[nb] = node
                frontier.append(nb)
    return None, expanded

def build_path(parent, node):
    path = []
    while node is not None:
        path.append(node); node = parent[node]
    return path[::-1]

S, G = find('S'), find('G')
print(f"{'Algorithm':<6}{'Path length':>13}{'Nodes expanded':>17}")
for name, algo in [('BFS', bfs), ('DFS', dfs)]:
    path, exp = algo(S, G)
    print(f"{name:<6}{len(path)-1:>13}{exp:>17}")

# ---------- Uniform Cost Search on a weighted graph ----------
graph = {
    'Islamabad':  {'Rawalpindi': 15, 'Murree': 60, 'Attock': 80},
    'Rawalpindi': {'Islamabad': 15, 'Chakwal': 90, 'Jhelum': 110},
    'Murree':     {'Islamabad': 60, 'Abbottabad': 65},
    'Attock':     {'Islamabad': 80, 'Peshawar': 100},
    'Chakwal':    {'Rawalpindi': 90, 'Lahore': 230},
    'Jhelum':     {'Rawalpindi': 110, 'Gujranwala': 110},
    'Gujranwala': {'Jhelum': 110, 'Lahore': 70},
    'Abbottabad': {'Murree': 65},
    'Peshawar':   {'Attock': 100},
    'Lahore':     {'Chakwal': 230, 'Gujranwala': 70},
}

def ucs(graph, start, goal):
    frontier = [(0, start, [start])]          # (path cost, node, path)
```

```
    explored = set(); expanded = 0
    while frontier:
        cost, node, path = heapq.heappop(frontier)   # lowest g(n) first
        if node in explored:
            continue
        explored.add(node); expanded += 1
        if node == goal:
            return path, cost, expanded
        for nb, w in graph[node].items():
            if nb not in explored:
                heapq.heappush(frontier, (cost + w, nb, path + [nb]))
    return None, float('inf'), expanded

path, cost, exp = ucs(graph, 'Islamabad', 'Lahore')
print("\nUCS path :", ' -> '.join(path))
print("UCS cost :", cost, "km | nodes expanded:", exp)
```

Output

```
Algorithm  Path length   Nodes expanded
BFS              10               23
DFS              16               18

UCS path : Islamabad -> Rawalpindi -> Jhelum -> Gujranwala -> Lahore
UCS cost : 305 km | nodes expanded: 10
```

Observation: BFS finds the shortest path (10 moves) but expands more nodes. DFS expands fewer nodes here but returns a longer, non-optimal path (16 moves). UCS returns the cheapest route (305 km) even though it is not the route with the fewest cities.

Lab Tasks

1. Modify `bfs` and `dfs` to also return the **maximum frontier size** and print a comparison table (path length, nodes expanded, max frontier size).
2. Print the maze with the path found by each algorithm marked with `*`.
3. Change the order of neighbours in `neighbours()` to (Left, Down, Right, Up). How does it affect the DFS path and the BFS path? Explain why.
4. Add the weighted edges Islamabad–Chakwal (120 km) and Chakwal–Gujranwala (150 km) to the graph. Run UCS again and report the new path and cost.

Post-Lab Task

Create a 10 × 10 maze from a text file (`maze.txt`), assign a cost of 1 to normal cells and 5 to “mud” cells (marked `m`), and compare BFS and UCS on it. Explain why the two paths differ.

Viva Questions

1. Which data structure does each algorithm use for its frontier?
2. When does UCS behave exactly like BFS?
3. Why is DFS not optimal?
4. What happens to DFS on a graph with cycles if we do not keep a visited set?

References

1. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Sections 3.3–3.4.
2. Python `heapq` documentation: https://docs.python.org/3/library/heapq.html

Summary

Students implemented BFS, DFS and UCS and compared them on a maze and a weighted road map, observing the trade-off between optimality and memory/time.

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab # 4: Depth-Limited and Iterative Deepening Search

Objective: To implement Depth-Limited Search and Iterative Deepening Depth-First Search, and to visualise their search patterns using graphical tools.

Week / Lab No.	:	Week 4 / Lab 4
CLO Mapping	:	CLO1, CLO2
Time Required	:	3 hrs
Programming Language	:	Python 3.x
Software Required	:	Anaconda (Jupyter Notebook) / Google Colab
Libraries	:	networkx, matplotlib

Learning Outcomes

- Implement depth-limited search and distinguish between *cutoff* and *failure*.
- Implement iterative deepening search and explain why it is complete and optimal.
- Visualise search trees using NetworkX and Matplotlib.

Theory

4.1 Depth-Limited Search (DLS)

DLS is DFS with a predetermined depth limit *l*: nodes at depth *l* are treated as if they have no successors. This solves the infinite-path problem of DFS, but if the shallowest goal is deeper than *l*, DLS will not find it. DLS returns three possible results: **solution**, **cutoff** (the limit was reached, a goal might exist deeper) or **failure** (no solution at all).

Pseudocode

```
function RECURSIVE-DLS(node, problem, limit) returns solution, failure or cutoff
    if GOAL-TEST(node.state) then return SOLUTION(node)
    else if limit = 0 then return cutoff
    cutoff_occurred <- false
    for each action in ACTIONS(node.state) do
        child  <- CHILD-NODE(problem, node, action)
        result <- RECURSIVE-DLS(child, problem, limit - 1)
        if result = cutoff then cutoff_occurred <- true
        else if result != failure then return result
    if cutoff_occurred then return cutoff else return failure
```

4.2 Iterative Deepening Depth-First Search (IDDFS)

IDDFS repeatedly applies DLS with limits 0, 1, 2, … until a goal is found. It combines the **low memory of DFS**, O(bd), with the **completeness and optimality of BFS** (for equal step costs). Although shallow nodes are regenerated many times, the overhead is small because most nodes are at the bottom level: the total number of generated nodes is still O(b^d).

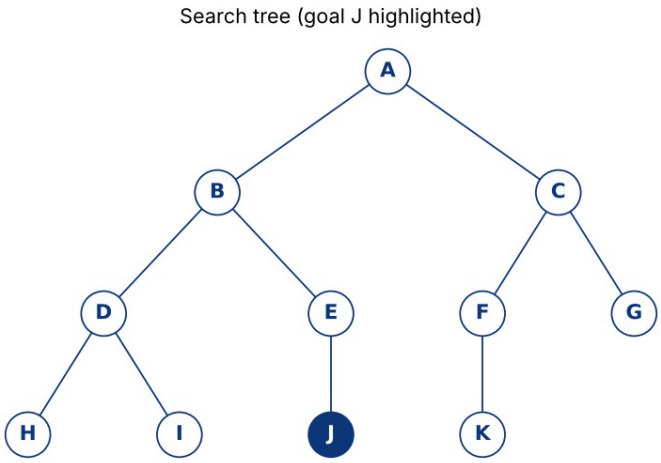


Figure 4.1: Search tree used in this lab (goal J highlighted)

Implementation

Example Code

```
import networkx as nx
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

tree = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': ['H', 'I'],
    'E': ['J'],
    'F': ['K'],
    'G': [],
    'H': [], 'I': [], 'J': [], 'K': []
}

def dls(node, goal, limit, visited_order):
    """Recursive Depth-Limited Search.
    Returns 'found', 'cutoff' or 'failure'."""
    visited_order.append(node)
    if node == goal:
        return 'found'
    if limit == 0:
        return 'cutoff'
    cutoff_occurred = False
    for child in tree[node]:
        res = dls(child, goal, limit - 1, visited_order)
        if res == 'found':
            return 'found'
        if res == 'cutoff':
            cutoff_occurred = True
    return 'cutoff' if cutoff_occurred else 'failure'

def iddfs(start, goal, max_depth=10):
    """Iterative Deepening: call DLS with limit 0, 1, 2, ..."""
    for depth in range(max_depth + 1):
        order = []
        res = dls(start, goal, depth, order)
        print(f"Depth limit {depth}: visited {' '.join(order):<24} -> {res}")
```

```
        if res == 'found':
            return depth
        if res == 'failure':
            return None
    return None

print("DLS with limit 2, goal = 'J'")
order = []
print("Result:", dls('A', 'J', 2, order), "| visited:", ' '.join(order))
print("\nIDDFS, goal = 'J'")
d = iddfs('A', 'J')
print("Goal found at depth", d)

# ----- Visualise the tree, colouring nodes by depth -----
G = nx.DiGraph([(p, c) for p, cs in tree.items() for c in cs])
# simple layered layout: each leaf gets its own x-slot, parents sit above children
pos, next_x = {}, [0]
def layout(n, d=0):
    kids = tree[n]
    if not kids:
        pos[n] = (next_x[0], -d); next_x[0] += 1
    else:
        for k in kids: layout(k, d + 1)
        pos[n] = (sum(pos[k][0] for k in kids) / len(kids), -d)
layout('A')
NAVY = '#0E367B'                        # Iqra University blue
colors = ['white' if n != 'J' else NAVY for n in G.nodes]
plt.figure(figsize=(6, 3.6))
nx.draw(G, pos, with_labels=False, node_color=colors, node_size=700,
        edgecolors=NAVY, edge_color=NAVY,
        arrows=False)
nx.draw_networkx_labels(G, pos, font_weight='bold',
        font_color={n: 'white' if n == 'J' else NAVY for n in G.nodes})
plt.title("Search tree (goal J highlighted)")
plt.savefig('lab04_tree.png', dpi=130, bbox_inches='tight')
print("Tree saved to lab04_tree.png")
```

Output

```
DLS with limit 2, goal = 'J'
Result: cutoff | visited: A B D E C F G

IDDFS, goal = 'J'
Depth limit 0: visited A                        -> cutoff
Depth limit 1: visited A B C                    -> cutoff
Depth limit 2: visited A B D E C F G            -> cutoff
Depth limit 3: visited A B D H I E J            -> found
Goal found at depth 3
Tree saved to lab04_tree.png
```

Observation: DLS with limit 2 returns *cutoff* because J is at depth 3. IDDFS increases the limit one level at a time and finds J at depth 3, the shallowest depth, just as BFS would.

Lab Tasks

1. Modify the program to count the **total** number of nodes visited by IDDFS across all iterations and compare it with the number of nodes visited by BFS on the same tree.

2. Visualise the search pattern: for each depth limit, save an image in which visited nodes are coloured green and unvisited nodes grey (use `nx.draw` with a colour list).
3. Apply IDDFS to the **8-puzzle** from Lab 2 for the start state (1, 2, 3, 4, 0, 6, 7, 5, 8). Print the depth at which the goal is found and the number of nodes expanded.
4. Implement DLS on the maze of Lab 3 with limits 5, 10 and 15 and report the result (found / cutoff) for each.

Post-Lab Task

Write a short comparison (one page) of BFS, DFS, DLS and IDDFS for the 8-puzzle using your own measurements: nodes expanded, maximum memory used and solution depth. Include a bar chart.

Viva Questions

1. What is the difference between cutoff and failure in DLS?
2. Why is the repeated work in IDDFS not a serious problem?
3. What is the space complexity of IDDFS?
4. When would you prefer IDDFS over BFS?

References

1. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Section 3.4.4–3.4.5.
2. NetworkX documentation: https://networkx.org/documentation/stable/

Summary

Students implemented depth-limited and iterative deepening search and visualised the search trees, learning how IDDFS achieves BFS-like optimality with DFS-like memory usage.

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab # 5: Informed Search: Greedy Best-First and A* Search

Objective: To understand heuristic search, implement Greedy Best-First Search and A* Search for the 8-puzzle using the Manhattan distance heuristic, and compare both on the same problem.

Week / Lab No.	:	Week 5 / Lab 5
CLO Mapping	:	CLO1, CLO2
Time Required	:	3 hrs
Programming Language	:	Python 3.x
Software Required	:	Anaconda (Jupyter Notebook) / Google Colab
Libraries	:	heapq, itertools

Learning Outcomes

- Define heuristic functions and the properties of admissibility and consistency.
- Implement Greedy Best-First Search and A* Search with a priority queue.
- Compare informed and uninformed search using node expansion counts.

Theory

5.1 Informed (Heuristic) Search

Informed search uses problem-specific knowledge in the form of a **heuristic function h(n)**: the estimated cost of the cheapest path from node *n* to a goal. Best-first search selects the node with the lowest evaluation function **f(n)** from the frontier.

5.2 Greedy Best-First Search

Greedy search expands the node that **appears** closest to the goal: **f(n) = h(n)**. It is often fast but is **neither optimal nor complete** (in tree search it may loop).

5.3 A* Search

A* combines the cost already paid and the estimated remaining cost: **f(n) = g(n) + h(n)**. A* is **complete and optimal** if *h(n)* is **admissible** (never overestimates the true cost) for tree search, or **consistent** ($h(n) \le c(n, a, n') + h(n')$) for graph search.

5.4 Heuristics for the 8-Puzzle

- **h1 = misplaced tiles:** the number of tiles not in their goal position.
- **h2 = Manhattan distance:** the sum of horizontal and vertical distances of each tile from its goal position. h2 dominates h1 ($h2 \ge h1$), so it leads to fewer expansions. Both are admissible.

For the start state of Figure 2.1: tile 8 is 3 moves from its goal, tile 1 is 1, tile 2 is 2, tile 6 is 2 and tile 5 is 2, while tiles 3, 4 and 7 are already in place. Therefore **h2 = 3 + 1 + 2 + 2 + 2 = 10**, while h1 (misplaced tiles) = 5.

Pseudocode

```
function A*-SEARCH(problem) returns a solution or failure
    frontier <- priority queue ordered by f = g + h, containing the initial node
    best_g[initial] <- 0
    while frontier is not empty do
        node <- POP(frontier)                          // lowest f(n)
        if GOAL-TEST(node.state) then return SOLUTION(node)
        for each child of node do
            g2 <- node.g + step_cost
            if child not in best_g or g2 < best_g[child] then
                best_g[child] <- g2
```

```
                insert child with f = g2 + h(child) into frontier
    return failure
```

Implementation

A single `best_first` function is used for both algorithms; only the evaluation function changes.

Example Code

```
import heapq, itertools

GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)
MOVES = {'Up': -3, 'Down': 3, 'Left': -1, 'Right': 1}

def neighbours(state):
    i = state.index(0); r, c = divmod(i, 3)
    for a, d in MOVES.items():
        if (a == 'Up' and r == 0) or (a == 'Down' and r == 2) \
           or (a == 'Left' and c == 0) or (a == 'Right' and c == 2):
            continue
        s = list(state); j = i + d
        s[i], s[j] = s[j], s[i]
        yield a, tuple(s)

def manhattan(state):
    """Sum of distances of each tile from its goal position."""
    dist = 0
    for idx, tile in enumerate(state):
        if tile == 0:
            continue
        goal_idx = tile - 1
        dist += abs(idx // 3 - goal_idx // 3) + abs(idx % 3 - goal_idx % 3)
    return dist

def best_first(start, f_type):
    """Generic best-first search.
    f_type = 'greedy' -> f(n) = h(n)
    f_type = 'astar'  -> f(n) = g(n) + h(n)"""
    counter = itertools.count()          # tie-breaker for the heap
    h0 = manhattan(start)
    frontier = [(h0, next(counter), 0, start, [])]
    best_g = {start: 0}
    expanded = 0
    while frontier:
        f, _, g, state, path = heapq.heappop(frontier)
        if state == GOAL:
            return path, expanded
        expanded += 1
        for action, child in neighbours(state):
            g2 = g + 1
            if child not in best_g or g2 < best_g[child]:
                best_g[child] = g2
                h = manhattan(child)
                f2 = h if f_type == 'greedy' else g2 + h
                heapq.heappush(frontier, (f2, next(counter), g2, child, path + [action]))
    return None, expanded

start = (8, 1, 3,
         4, 0, 2,
```

```
        7, 6, 5)
print("Start h(n) =", manhattan(start))
print(f"{'Algorithm':<10}{'Solution length':>17}{'Nodes expanded':>17}")
for name in ['greedy', 'astar']:
    path, exp = best_first(start, name)
    print(f"{name:<10}{len(path):>17}{exp:>17}")
path, _ = best_first(start, 'astar')
print("A* moves:", ' '.join(path))
```

Output

```
Start h(n) = 10
Algorithm   Solution length   Nodes expanded
greedy                  36              181
astar                   14               76
A* moves: Right Down Left Left Up Up Right Down Left Down Right Up Right Down
```

Observation: For the same problem, BFS (Lab 2) needs about **5,960** expansions to find the 14-move optimal solution. A* finds the same optimal solution with only **76** expansions. Greedy search expands more nodes here and returns a much longer, non-optimal 36-move solution.

Lab Tasks

1. Implement the **misplaced tiles** heuristic h1 and run A* with h1 and with h2 on the same start state. Compare nodes expanded.
2. Create a comparison table for 5 start states of increasing difficulty with columns: BFS, Greedy (h2), A* (h1), A* (h2) – nodes expanded and solution length.
3. Prove (by a short argument) that the Manhattan distance is admissible for the 8-puzzle.
4. Apply A* to the **maze** of Lab 3 using the Manhattan distance between a cell and G as h(n). Compare its node expansions with BFS.

Post-Lab Task

Implement A* to find the shortest road route between two cities of Pakistan using the graph of Lab 3, with the straight-line (air) distance to Lahore as the heuristic. Look up approximate straight-line distances and check that your heuristic is admissible.

Viva Questions

1. What is the difference between greedy search and A*?
2. What does it mean for a heuristic to be admissible? consistent?
3. Why does a dominating heuristic expand fewer nodes?
4. What happens to A* if h(n) = 0 for all n?

References

1. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Section 3.5–3.6.
2. Red Blob Games, “Introduction to A*”: https://www.redblobgames.com/pathfinding/a-star/introduction.html

Summary

Students implemented Greedy Best-First and A* search with the Manhattan distance heuristic and saw how a good admissible heuristic dramatically reduces the search effort while preserving optimality.

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab # 6: Local Search and Optimization: Hill Climbing and Genetic Algorithm

Objective: To explore local search and optimization techniques by implementing Hill Climbing for the 8-queens problem and a Genetic Algorithm to optimize a mathematical function.

Week / Lab No.	:	Week 6 / Lab 6
CLO Mapping	:	CLO1, CLO2
Time Required	:	3 hrs
Programming Language	:	Python 3.x
Software Required	:	Anaconda (Jupyter Notebook) / Google Colab
Libraries	:	random, math

Learning Outcomes

- Explain local search, objective functions and the state-space landscape.
- Implement steepest-ascent hill climbing and random-restart hill climbing for 8-queens.
- Implement a genetic algorithm with selection, crossover, mutation and elitism.

Part A: Hill Climbing

6.1 Local Search

In many problems the **path to the goal is irrelevant**; only the final state matters (e.g., 8-queens, timetabling, VLSI layout). Local search algorithms keep a single current state and move to neighbouring states. They use very little memory and can find reasonable solutions in large or continuous spaces.

6.2 Hill Climbing

Steepest-ascent hill climbing moves to the best neighbour and stops when no neighbour is better. It can get stuck on **local maxima**, **ridges** and **plateaux**. **Random-restart hill climbing** runs hill climbing from many random initial states until a solution is found, making it complete with probability approaching 1.

6.3 The 8-Queens Problem

Place eight queens on a chess board so that no two attack each other. **State:** `board[c]` = row of the queen in column *c*. **Neighbours:** move one queen to another row in its column (8 × 7 = 56 neighbours). **Heuristic h:** number of pairs of queens attacking each other (goal: h = 0).

Pseudocode

```
function HILL-CLIMBING(problem) returns a state that is a local optimum
    current <- problem.INITIAL-STATE
    loop do
        neighbour <- best successor of current (lowest h)
        if h(neighbour) >= h(current) then return current
        current <- neighbour
```

Example Code

```
import random

N = 8

def attacking_pairs(board):
    """board[c] = row of the queen in column c. Count pairs that attack."""
    h = 0
```

```
    for c1 in range(N):
        for c2 in range(c1 + 1, N):
            if board[c1] == board[c2] or abs(board[c1] - board[c2]) == c2 - c1:
                h += 1
    return h

def best_neighbour(board):
    """Try moving each queen to every other row in its column."""
    best, best_h = None, attacking_pairs(board)
    for col in range(N):
        for row in range(N):
            if row == board[col]:
                continue
            nb = list(board); nb[col] = row
            h = attacking_pairs(nb)
            if h < best_h:
                best, best_h = nb, h
    return best, best_h

def hill_climbing(board):
    steps = 0
    while True:
        nb, h = best_neighbour(board)
        if nb is None:                  # no better neighbour -> local optimum
            return board, attacking_pairs(board), steps
        board, steps = nb, steps + 1

def random_restart(max_restarts=100):
    for r in range(max_restarts):
        start = [random.randrange(N) for _ in range(N)]
        board, h, steps = hill_climbing(start)
        if h == 0:
            return board, r + 1
    return None, max_restarts

random.seed(3)
start = [random.randrange(N) for _ in range(N)]
board, h, steps = hill_climbing(start)
print("Start board :", start, "h =", attacking_pairs(start))
print("Hill climb  :", board, "h =", h, "after", steps, "steps")

# Success rate of plain hill climbing over 200 random boards
wins = sum(hill_climbing([random.randrange(N) for _ in range(N)])[1] == 0 for _ in
range(200))
print(f"Plain hill climbing success rate: {wins/200:.0%}")

sol, restarts = random_restart()
print("Random-restart solution:", sol, "found after", restarts, "restart(s)")
for r in range(N):
    print(' '.join('Q' if sol[c] == r else '.' for c in range(N)))
```

Output

```
Start board : [3, 2, 5, 7, 1, 0, 7, 4] h = 6
Hill climb  : [6, 3, 5, 7, 2, 0, 7, 4] h = 2 after 3 steps
Plain hill climbing success rate: 14%
Random-restart solution: [3, 7, 0, 2, 5, 1, 6, 4] found after 5 restart(s)
. . Q . . . . .
. . . . . Q . .
```

```
. . . Q . . . .
Q . . . . . . .
. . . . . . . Q
. . . . Q . . .
. . . . . . Q .
. Q . . . . . .
```

Observation: Plain hill climbing solves only about 14% of random boards (it usually gets stuck at a local minimum). Random restarts overcome this limitation.

Part B: Genetic Algorithm

6.4 Genetic Algorithm (GA)

A genetic algorithm is a population-based search inspired by natural selection. Each candidate solution is encoded as a **chromosome** (e.g., a binary string). A **fitness function** rates each chromosome, and new generations are produced by **selection**, **crossover** and **mutation**.

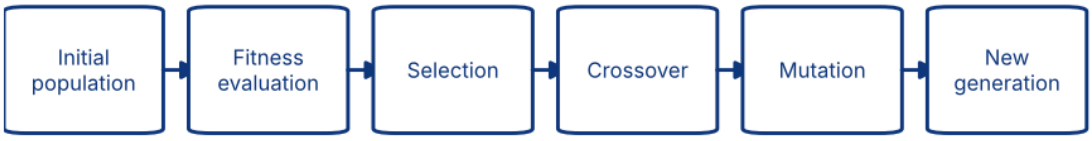


Figure 6.1: The genetic algorithm cycle

In this example we maximise **f(x) = x · sin(10πx) + 1** on the interval [−1, 2]. The function has many local maxima, which makes it hard for hill climbing. Each chromosome is a 22-bit binary string decoded to a real number. We use tournament selection, single-point crossover (rate 0.9), bit-flip mutation (rate 1/22) and elitism.

Example Code

```
import random, math

# Maximise f(x) = x * sin(10*pi*x) + 1   on the interval [-1, 2]
def f(x):
    return x * math.sin(10 * math.pi * x) + 1

BITS, LOW, HIGH = 22, -1.0, 2.0

def decode(chrom):
    """Binary string -> real number in [LOW, HIGH]."""
    return LOW + int(chrom, 2) * (HIGH - LOW) / (2**BITS - 1)

def random_chrom():
    return ''.join(random.choice('01') for _ in range(BITS))

def tournament(pop, k=3):
    return max(random.sample(pop, k), key=lambda c: f(decode(c)))

def crossover(p1, p2, rate=0.9):
    if random.random() < rate:
        pt = random.randint(1, BITS - 1)          # single-point crossover
        return p1[:pt] + p2[pt:], p2[:pt] + p1[pt:]
    return p1, p2
```

```
def mutate(c, rate=1/BITS):
    return ''.join(b if random.random() > rate else '10'[int(b)] for b in c)

def genetic_algorithm(pop_size=50, generations=60):
    pop = [random_chrom() for _ in range(pop_size)]
    for gen in range(generations + 1):
        best = max(pop, key=lambda c: f(decode(c)))
        if gen % 10 == 0:
            print(f"Gen {gen:3d}: best x = {decode(best):.5f}, f(x) =
{f(decode(best)):.5f}")
        new_pop = [best]                          # elitism
        while len(new_pop) < pop_size:
            c1, c2 = crossover(tournament(pop), tournament(pop))
            new_pop += [mutate(c1), mutate(c2)]
        pop = new_pop[:pop_size]
    return best

random.seed(42)
best = genetic_algorithm()
print(f"Best solution: x = {decode(best):.5f}, f(x) = {f(decode(best)):.5f}")
```

Output

```
Gen   0: best x = 1.66575, f(x) = 2.46594
Gen  10: best x = 1.85064, f(x) = 2.85027
Gen  20: best x = 1.85059, f(x) = 2.85027
Gen  30: best x = 1.85059, f(x) = 2.85027
Gen  40: best x = 1.85059, f(x) = 2.85027
Gen  50: best x = 1.85059, f(x) = 2.85027
Gen  60: best x = 1.85059, f(x) = 2.85027
Best solution: x = 1.85059, f(x) = 2.85027
```

Observation: The GA reaches $x \approx 1.8506$ with $f(x) \approx 2.8503$, which is the global maximum of the function on this interval.

Lab Tasks

1. Implement **first-choice hill climbing** (move to the first better neighbour found randomly) for 8-queens and compare its success rate with steepest-ascent over 200 runs.
2. Add **sideways moves** (allow up to 100 consecutive moves with equal h) and report the new success rate.
3. Run the GA with mutation rates 0, 0.01, 0.05 and 0.2. Plot the best fitness against generation for each rate and explain the effect of mutation.
4. Solve the 8-queens problem using the GA: a chromosome is a list of 8 rows, and fitness = 28 − attacking pairs. Report the generation in which a solution is found.

Post-Lab Task

Implement **simulated annealing** for the 8-queens problem with a geometric cooling schedule and compare it with random-restart hill climbing in terms of success rate and average running time.

Viva Questions

1. Why is hill climbing called a greedy local search?
2. What are local maxima, plateaux and ridges?
3. What is the role of crossover and of mutation in a GA?

4. Why is elitism used in a GA?

References

1. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Section 4.1.
2. D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*, Addison-Wesley.

Summary

Students implemented hill climbing with random restarts for 8-queens and a genetic algorithm for function optimisation, and studied the strengths and weaknesses of local search.

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Open-Ended Lab / Project: Smart Campus Delivery Robot

Objective: To apply AI techniques learned in the course to an open-ended, real-world problem that has no single correct solution, and to design, implement, evaluate and justify an intelligent agent.

Week / Lab No.	:	Assigned in Week 7, assessed in Weeks 15–16
CLO Mapping	:	CLO1, CLO2
Time Required	:	3 hrs
Programming Language	:	Python 3.x
Software Required	:	Anaconda (Jupyter Notebook) / Google Colab
Libraries	:	Any (justify your choice)

Group Formation

Students will work in groups of **2–3 members**. Each member must contribute to design, coding and the report, and must be able to explain every part of the system during the viva.

Problem Statement

The university plans to use a small autonomous robot to deliver documents and parcels between buildings on campus (Admin Block, Library, CS Department, Cafeteria, Labs, Hostel and Main Gate). The campus is represented as a grid map with buildings, footpaths, grass areas (slower), stairs (not allowed for the robot), and temporary obstacles (e.g., construction). Different paths have different costs depending on distance, surface type and congestion at different times of the day.

Your group must design an **intelligent delivery agent** that receives a list of delivery requests and plans how to complete them efficiently and safely.

Tasks

1. **Agent design (CLO1):** Write the PEAS description of the delivery robot and classify its task environment. Justify which agent type is appropriate.
2. **Problem formulation (CLO1):** Create a campus map (at least 15 × 15 grid) with at least 7 locations. Define states, actions, transition model, goal test and a path cost function that you design and justify.
3. **Path planning (CLO2):** Implement BFS, UCS and A* for a single delivery. Design **at least two admissible heuristics** and compare all algorithms (nodes expanded, path cost, time) in a table and chart.
4. **Multiple deliveries (CLO2):** Given 4–6 delivery requests, find a good visiting order. Use either a local search (hill climbing/simulated annealing) or a genetic algorithm, and compare it with a brute-force or greedy baseline.
5. **Intelligent extension (choose ONE):** (a) a fuzzy controller that sets the robot's speed from crowd density and battery level; (b) an ML model (e.g., Naive Bayes or MLP) that predicts path congestion from time of day and day of week using a synthetic dataset; (c) dynamic re-planning when a new obstacle appears.
6. **Visualisation:** Display the map, the planned routes and the robot's movement (Matplotlib, Pygame or a simple GUI).

Constraints

- The robot cannot pass through buildings, stairs or obstacle cells.
- Battery capacity is limited; each move consumes energy (more on grass). The robot must return to the charging station at the Main Gate if the battery falls below 20%.

- The solution must run in under one minute for the largest test case.

Deliverables

Deliverable	Details	Due
Proposal (1 page)	Group members, chosen extension, initial formulation, work division	Week 9
Source code	Well-commented code with README (how to run), uploaded to LMS / GitHub	Week 15
Technical report (6–10 pages)	Formulation, algorithms, heuristics and their admissibility, experiments, results, discussion, limitations and individual contributions (use Appendix B template)	Week 15
Demonstration and viva	10-minute live demo followed by individual questions	Weeks 15–16

Assessment

The OEL is assessed using **Lab Rubrics (Group 2)**: R1 Problem Formulation and Design, R2 Implementation and Results, R3 Demonstration and Viva, R4 Contribution / Group Participation, and R5 Report and Presentation (each 0–3, total 15).

Complex Problem Attributes

- **Depth of knowledge:** requires search, optimisation, fuzzy logic or machine learning.
- **Conflicting requirements:** shortest time vs energy consumption vs safety.
- **No obvious solution:** heuristics, cost function and extension must be designed and justified by the group.
- **Range of stakeholders:** students, staff, campus security.

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 2). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab # 7: Adversarial Search: Minimax and Alpha-Beta Pruning

Objective: To understand game-playing agents, implement the Minimax algorithm for Tic-Tac-Toe, and enhance its performance using Alpha-Beta pruning.

Week / Lab No.	:	Week 9 / Lab 7
CLO Mapping	:	CLO1, CLO2
Time Required	:	3 hrs
Programming Language	:	Python 3.x
Software Required	:	Anaconda (Jupyter Notebook) / Google Colab
Libraries	:	math

Learning Outcomes

- Formulate a two-player, zero-sum game as a search problem.
- Implement Minimax and compute minimax values of a game tree.
- Implement Alpha-Beta pruning and measure the reduction in searched nodes.

Theory

7.1 Games as Search Problems

In a **two-player, zero-sum, deterministic game with perfect information** (e.g., Tic-Tac-Toe, chess), the players are called **MAX** and **MIN**. A game is defined by: the initial state, `PLAYER(s)`, `ACTIONS(s)`, `RESULT(s, a)`, `TERMINAL-TEST(s)` and `UTILITY(s, p)`. In Tic-Tac-Toe, utility is +1 if X (MAX) wins, −1 if O (MIN) wins and 0 for a draw.

7.2 The Minimax Algorithm

Minimax computes the **minimax value** of each node: the utility of a terminal state, the maximum of the children's values at a MAX node, and the minimum at a MIN node. It assumes both players play optimally. Minimax explores the complete game tree: time O(b^m), space O(bm).

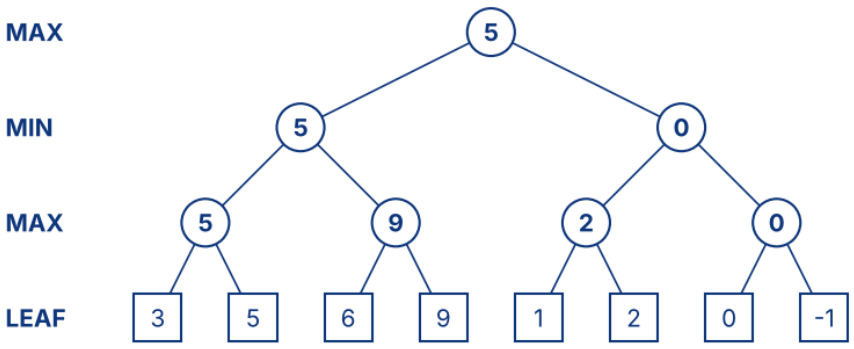


Figure 7.1: A game tree with minimax values backed up from the leaves

Pseudocode

```
function MINIMAX(state, isMax) returns a utility value
    if TERMINAL-TEST(state) then return UTILITY(state)
    if isMax then
        return max over actions a of MINIMAX(RESULT(state, a), false)
    else
        return min over actions a of MINIMAX(RESULT(state, a), true)
```

7.3 Alpha-Beta Pruning

Alpha-Beta pruning returns the **same move** as Minimax but prunes branches that cannot influence the final decision. **α** is the best value found so far for MAX along the path, and **β** is the best value found so far for MIN. Whenever α ≥ β, the remaining children of the node are pruned. With perfect move ordering the time complexity becomes O(b^(m/2)), allowing search twice as deep.

Pseudocode

```
function ALPHA-BETA(state, isMax, alpha, beta)
    if TERMINAL-TEST(state) then return UTILITY(state)
    if isMax then
        v <- -infinity
        for each a in ACTIONS(state) do
            v <- max(v, ALPHA-BETA(RESULT(state, a), false, alpha, beta))
            alpha <- max(alpha, v)
            if alpha >= beta then break          // beta cut-off
        return v
    else  (symmetric for MIN, updating beta, alpha cut-off)
```

Implementation

Example Code

```
import math

WIN_LINES = [(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]

def winner(b):
    for a, c, d in WIN_LINES:
        if b[a] != ' ' and b[a] == b[c] == b[d]:
            return b[a]
    return None

def terminal(b):
    return winner(b) is not None or ' ' not in b

def utility(b):
    w = winner(b)
    return 1 if w == 'X' else -1 if w == 'O' else 0   # X = MAX, O = MIN

nodes = 0

def minimax(b, is_max):
    global nodes; nodes += 1
    if terminal(b):
        return utility(b)
    scores = []
    for i in range(9):
        if b[i] == ' ':
            b[i] = 'X' if is_max else 'O'
            scores.append(minimax(b, not is_max))
            b[i] = ' '                                  # undo move
    return max(scores) if is_max else min(scores)

def alphabeta(b, is_max, alpha=-math.inf, beta=math.inf):
    global nodes; nodes += 1
    if terminal(b):
        return utility(b)
    if is_max:
```

```
        value = -math.inf
        for i in range(9):
            if b[i] == ' ':
                b[i] = 'X'
                value = max(value, alphabeta(b, False, alpha, beta))
                b[i] = ' '
                alpha = max(alpha, value)
                if alpha >= beta:
                    break                                  # beta cut-off
        return value
    else:
        value = math.inf
        for i in range(9):
            if b[i] == ' ':
                b[i] = 'O'
                value = min(value, alphabeta(b, True, alpha, beta))
                b[i] = ' '
                beta = min(beta, value)
                if alpha >= beta:
                    break                                  # alpha cut-off
        return value

def best_move(b, player, search):
    best_i, best_v = None, (-math.inf if player == 'X' else math.inf)
    for i in range(9):
        if b[i] == ' ':
            b[i] = player
            v = search(b, player == 'O')
            b[i] = ' '
            if (player == 'X' and v > best_v) or (player == 'O' and v < best_v):
                best_i, best_v = i, v
    return best_i, best_v

def show(b):
    for r in range(3):
        print(' ' + ' | '.join(b[r*3:r*3+3]))
        if r < 2: print('---+---+---')

# Compare the two algorithms on the empty board
for name, algo in [('Minimax', minimax), ('Alpha-Beta', alphabeta)]:
    nodes = 0
    move, val = best_move([' '] * 9, 'X', algo)
    print(f"{name:<11} best first move = {move}, value = {val}, nodes searched = {nodes}")

# AI (X) vs AI (O) game using alpha-beta: perfect play ends in a draw
board = [' '] * 9; player = 'X'
while not terminal(board):
    move, _ = best_move(board, player, alphabeta)
    board[move] = player
    player = 'O' if player == 'X' else 'X'
show(board)
print("Result:", winner(board) or "Draw")
```

Output

```
Minimax     best first move = 0, value = 0, nodes searched = 549945
Alpha-Beta  best first move = 0, value = 0, nodes searched = 30709
 X | X | O
---+---+---
```

```
 O | O | X
---+---+---
 X | O | X
Result: Draw
```

Observation: Both algorithms choose the same first move with the same value (0 = draw with perfect play), but Alpha-Beta searches about **30,709** nodes instead of **549,945** – a reduction of over 94%.

Lab Tasks

1. Make the game **interactive**: the human plays O by entering a cell number (0–8) and the AI plays X using Alpha-Beta. Validate the user input.
2. Compute by hand the minimax value of the tree in Figure 7.1 and mark which leaves would be pruned by Alpha-Beta (left-to-right order). Verify with code.
3. Improve move ordering: try the centre first, then corners, then edges. Report the new node count for Alpha-Beta on the empty board.
4. Add a **depth limit** and an evaluation function (number of open lines for X minus open lines for O). Compare the moves chosen with depth limits 2, 4 and 9.

Post-Lab Task

Implement Alpha-Beta search for **Connect-4** (6 × 7 board) with a depth limit of 5 and a heuristic evaluation function of your own design. Let your AI play against a random player for 20 games and report the win rate.

Viva Questions

1. Why is Tic-Tac-Toe called a zero-sum game?
2. Does Alpha-Beta pruning ever change the result of Minimax?
3. Why does move ordering matter in Alpha-Beta?
4. What is an evaluation function and when do we need it?

References

1. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Chapter 5.
2. GeeksforGeeks, “Minimax Algorithm in Game Theory” series.

Summary

Students built an unbeatable Tic-Tac-Toe agent using Minimax and observed how Alpha-Beta pruning returns the same decision while searching far fewer nodes.

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab # 8: Fuzzy Sets and Fuzzy Set Operations

Objective: To represent crisp and fuzzy sets in Python and to perform union, intersection and complement operations on fuzzy sets.

Week / Lab No.	:	Week 10 / Lab 8
CLO Mapping	:	CLO1
Time Required	:	3 hrs
Programming Language	:	Python 3.x
Software Required	:	Anaconda (Jupyter Notebook) / Google Colab
Libraries	:	numpy, matplotlib

Learning Outcomes

- Differentiate between crisp (classical) sets and fuzzy sets.
- Implement triangular and trapezoidal membership functions.
- Perform and visualise standard fuzzy set operations.

Theory

8.1 Crisp Sets vs Fuzzy Sets

In a **crisp set**, an element either belongs to the set (membership 1) or not (membership 0). For example, if HOT = {t | t ≥ 25 °C}, then 24.9 °C is not hot at all while 25 °C is completely hot. A **fuzzy set** A on a universe X is defined by a **membership function** μ_A : X → [0, 1] that gives each element a **degree of membership**. Fuzzy logic, introduced by Lotfi Zadeh (1965), models vague linguistic terms such as *warm*, *tall* or *fast*.

8.2 Membership Functions

- **Triangular:** trimf(x; a, b, c) = max(min((x − a)/(b − a), (c − x)/(c − b)), 0)
- **Trapezoidal:** trapmf(x; a, b, c, d) = max(min((x − a)/(b − a), 1, (d − x)/(d − c)), 0)
- **Gaussian:** gaussmf(x; c, σ) = exp(−(x − c)² / 2σ²)

8.3 Standard Fuzzy Set Operations (Zadeh operators)

Operation	Definition	Linguistic meaning
Union A ∪ B	μ(x) = max(μ_A(x), μ_B(x))	A OR B
Intersection A ∩ B	μ(x) = min(μ_A(x), μ_B(x))	A AND B
Complement A′	μ(x) = 1 − μ_A(x)	NOT A
Algebraic product	μ(x) = μ_A(x) · μ_B(x)	alternative AND
Algebraic sum	μ(x) = μ_A + μ_B − μ_A·μ_B	alternative OR

Note that fuzzy sets **do not** satisfy the law of excluded middle: A ∪ A′ ≠ X and A ∩ A′ ≠ ∅ in general.

Implementation

Example Code

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
```

```
# ---------- Crisp set ----------
temperatures = [10, 18, 22, 27, 35]
hot_crisp = {t for t in temperatures if t >= 25}
print("Crisp set HOT (t >= 25):", sorted(hot_crisp))

# ---------- Discrete fuzzy sets (element: membership degree) ----------
A = {'x1': 0.2, 'x2': 0.7, 'x3': 1.0, 'x4': 0.4}
B = {'x1': 0.5, 'x2': 0.3, 'x3': 0.8, 'x4': 0.9}

union        = {x: max(A[x], B[x]) for x in A}
intersection = {x: min(A[x], B[x]) for x in A}
complement_A = {x: round(1 - A[x], 2) for x in A}

print("A           :", A)
print("B           :", B)
print("A union B   :", union)
print("A inter B   :", intersection)
print("Complement A:", complement_A)

# ---------- Continuous membership functions ----------
def trimf(x, a, b, c):
    """Triangular membership function."""
    return np.maximum(np.minimum((x - a) / (b - a), (c - x) / (c - b)), 0)

def trapmf(x, a, b, c, d):
    """Trapezoidal membership function."""
    return np.maximum(np.minimum(np.minimum((x - a) / (b - a + 1e-9), 1),
                                 (d - x) / (d - c + 1e-9)), 0)

x = np.linspace(0, 40, 401)
cold = trapmf(x, -1, 0, 10, 20)
warm = trimf(x, 15, 22.5, 30)
hot  = trapmf(x, 25, 32, 40, 41)

print("\nMembership of 27 C -> cold: %.2f, warm: %.2f, hot: %.2f"
      % (cold[270], warm[270], hot[270]))

plt.figure(figsize=(7, 3.2))
NAVY = '#0E367B'                        # Iqra University blue
plt.plot(x, cold, color=NAVY, ls=':', label='Cold')
plt.plot(x, warm, color=NAVY, ls='--', label='Warm')
plt.plot(x, hot, color=NAVY, ls='-', label='Hot')
plt.fill_between(x, np.fmax(warm, hot), color=NAVY, alpha=0.12, label='Warm OR Hot')
plt.fill_between(x, np.fmin(warm, hot), color=NAVY, alpha=0.6, label='Warm AND Hot')
plt.xlabel('Temperature (C)'); plt.ylabel('Membership'); plt.legend(fontsize=8)
plt.title('Fuzzy sets for temperature')
plt.savefig('lab08_fuzzy.png', dpi=130, bbox_inches='tight')
```

Output

```
Crisp set HOT (t >= 25): [27, 35]
A           : {'x1': 0.2, 'x2': 0.7, 'x3': 1.0, 'x4': 0.4}
B           : {'x1': 0.5, 'x2': 0.3, 'x3': 0.8, 'x4': 0.9}
A union B   : {'x1': 0.5, 'x2': 0.7, 'x3': 1.0, 'x4': 0.9}
A inter B   : {'x1': 0.2, 'x2': 0.3, 'x3': 0.8, 'x4': 0.4}
Complement A: {'x1': 0.8, 'x2': 0.3, 'x3': 0.0, 'x4': 0.6}

Membership of 27 C -> cold: 0.00, warm: 0.40, hot: 0.29
```

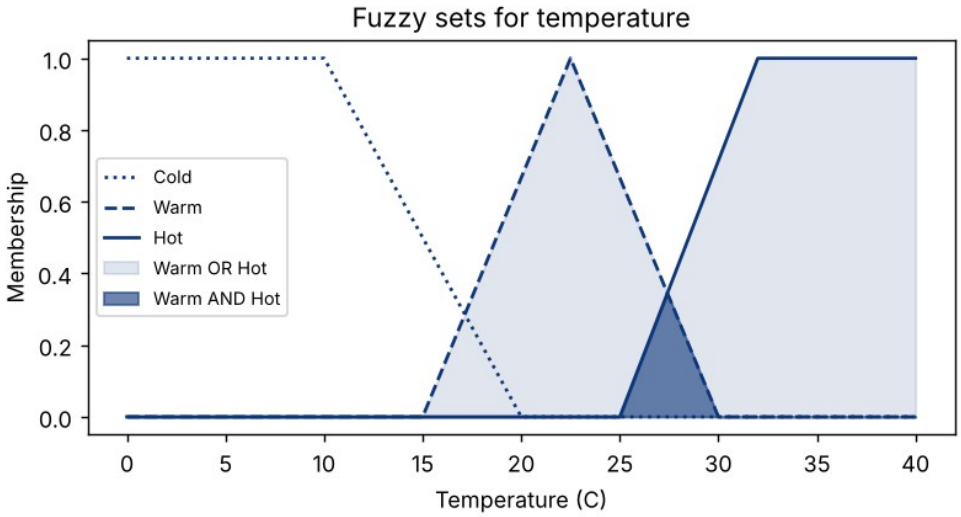


Figure 8.1: Fuzzy sets Cold, Warm and Hot with their union and intersection

Lab Tasks

1. Write functions `fuzzy_union`, `fuzzy_intersection`, `fuzzy_complement`, `algebraic_product` and `algebraic_sum` that work on dictionaries, and test them on sets A and B.
2. Verify **De Morgan's laws** for fuzzy sets A and B: $(A \cup B)' = A' \cap B'$ and $(A \cap B)' = A' \cup B'$.
3. Show numerically that $A \cup A' \neq X$ for set A. Explain why this happens.
4. Define fuzzy sets **Young**, **Middle-aged** and **Old** for age (0–100 years) using Gaussian membership functions. Plot them and find the membership of a 35-year-old person in each set.

Post-Lab Task

Implement the **fuzzy Cartesian product** and **max–min composition** of two fuzzy relations R (X × Y) and S (Y × Z) using NumPy, and test them on 3 × 3 relation matrices.

Viva Questions

1. What is the difference between probability and fuzzy membership?
2. Why is min used for fuzzy AND and max for fuzzy OR?
3. What is the support and the core of a fuzzy set?
4. Give two real-world applications of fuzzy logic.

References

1. L. A. Zadeh, "Fuzzy Sets," *Information and Control*, vol. 8, pp. 338–353, 1965.
2. T. J. Ross, *Fuzzy Logic with Engineering Applications*, 4th ed., Wiley, 2016.

Summary

Students represented crisp and fuzzy sets, implemented membership functions and fuzzy set operations, and visualised them with Matplotlib.

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab # 9: Fuzzy Logic Based Expert System (AC Temperature Control)

Objective: To design a fuzzy logic-based expert system for air-conditioner control, implementing fuzzification, rule evaluation, aggregation and defuzzification.

Week / Lab No.	:	Week 11 / Lab 9
CLO Mapping	:	CLO1, CLO2
Time Required	:	3 hrs
Programming Language	:	Python 3.x
Software Required	:	Anaconda (Jupyter Notebook) / Google Colab
Libraries	:	numpy (optional: scikit-fuzzy)

Learning Outcomes

- Describe the architecture of a fuzzy inference system (FIS).
- Design linguistic variables, membership functions and a fuzzy rule base.
- Implement Mamdani inference with centroid defuzzification from scratch.

Theory

9.1 Fuzzy Inference System

A fuzzy expert system maps crisp inputs to crisp outputs using IF–THEN rules expressed in linguistic terms. The **Mamdani** fuzzy inference process has four steps:

1. **Fuzzification:** convert each crisp input into degrees of membership of its linguistic terms.
2. **Rule evaluation:** compute the firing strength of each rule. For `AND` use min, for `OR` use max. The output set of the rule is clipped at the firing strength (implication).
3. **Aggregation:** combine the clipped output sets of all rules using max.
4. **Defuzzification:** convert the aggregated fuzzy set into one crisp value. The **centroid** method is the most common: $y^* = \Sigma\, y \cdot \mu(y) \,/\, \Sigma\, \mu(y)$.

9.2 Problem: AC Cooling Power Controller

An air conditioner adjusts its cooling power according to the room **temperature** (10–45 °C) and **humidity** (0–100%). The output is the **cooling power** (0–100%).

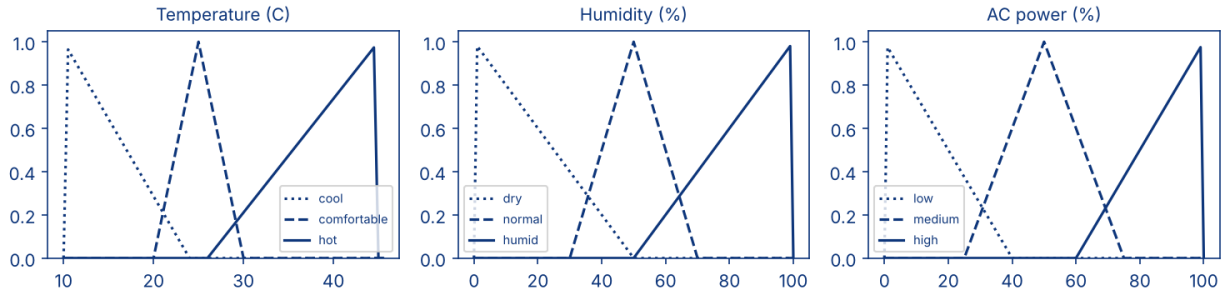


Figure 9.1: Membership functions of the input and output variables

Temperature \ Humidity	Dry	Normal	Humid
Cool	Low	Low	Medium
Comfortable	Low	Medium	High
Hot	Medium	High	High

Table 9.1: Fuzzy rule base (fuzzy associative memory). Example: IF temperature is Hot AND humidity is Normal THEN power is High.

Implementation

Example Code

```
import numpy as np

def trimf(x, a, b, c):
    return np.maximum(np.minimum((x - a) / (b - a + 1e-9), (c - x) / (c - b + 1e-9)), 0)

# ---------- Universes of discourse ----------
temp_u  = np.arange(10, 46, 0.5)     # room temperature in C
hum_u   = np.arange(0, 101, 1)       # relative humidity in %
power_u = np.arange(0, 101, 1)       # AC cooling power in %

# ---------- Fuzzy sets (linguistic terms) ----------
temp_sets  = {'cool': (10, 10, 24), 'comfortable': (20, 25, 30), 'hot': (26, 45, 45)}
hum_sets   = {'dry': (0, 0, 50), 'normal': (30, 50, 70), 'humid': (50, 100, 100)}
power_sets = {'low': (0, 0, 40), 'medium': (25, 50, 75), 'high': (60, 100, 100)}

def fuzzify(value, sets):
    return {name: float(trimf(np.array([value]), *abc)[0]) for name, abc in sets.items()}

# ---------- Rule base (IF temp AND humidity THEN power) ----------
rules = [
    ('cool',        'dry',    'low'),
    ('cool',        'normal', 'low'),
    ('cool',        'humid',  'medium'),
    ('comfortable', 'dry',    'low'),
    ('comfortable', 'normal', 'medium'),
    ('comfortable', 'humid',  'high'),
    ('hot',         'dry',    'medium'),
    ('hot',         'normal', 'high'),
    ('hot',         'humid',  'high'),
]

def ac_controller(temperature, humidity, verbose=True):
    # 1. Fuzzification
    t = fuzzify(temperature, temp_sets)
    h = fuzzify(humidity, hum_sets)
    if verbose:
        print("Fuzzified temperature:", {k: round(v, 2) for k, v in t.items()})
        print("Fuzzified humidity   :", {k: round(v, 2) for k, v in h.items()})
    # 2. Rule evaluation (AND = min) and implication (clip = min)
    aggregated = np.zeros_like(power_u, dtype=float)
    for ts, hs, ps in rules:
        strength = min(t[ts], h[hs])
        if strength > 0:
            if verbose:
                print(f"  Rule fired: IF temp is {ts} AND humidity is {hs} "
                      f"THEN power is {ps}  (strength {strength:.2f})")
            clipped = np.fmin(strength, trimf(power_u, *power_sets[ps]))
            # 3. Aggregation (OR = max)
            aggregated = np.fmax(aggregated, clipped)
    # 4. Defuzzification (centroid)
    if aggregated.sum() == 0:
        return 0.0
    return float((power_u * aggregated).sum() / aggregated.sum())
```

```
out = ac_controller(29, 65)
print(f"Crisp output -> AC cooling power = {out:.1f}%\n")

print("Temperature  Humidity  Power")
for T, H in [(18, 30), (24, 50), (32, 40), (38, 85)]:
    print(f"{T:>8} C {H:>8} % {ac_controller(T, H, verbose=False):>7.1f} %")
```

Output

```
Fuzzified temperature: {'cool': 0.0, 'comfortable': 0.2, 'hot': 0.16}
Fuzzified humidity   : {'dry': 0.0, 'normal': 0.25, 'humid': 0.3}
  Rule fired: IF temp is comfortable AND humidity is normal THEN power is medium  (strength
0.20)
  Rule fired: IF temp is comfortable AND humidity is humid THEN power is high  (strength
0.20)
  Rule fired: IF temp is hot AND humidity is normal THEN power is high  (strength 0.16)
  Rule fired: IF temp is hot AND humidity is humid THEN power is high  (strength 0.16)
Crisp output -> AC cooling power = 63.5%

Temperature  Humidity  Power
      18 C       30 %    16.6 %
      24 C       50 %    50.0 %
      32 C       40 %    67.6 %
      38 C       85 %    85.1 %
```

Observation: At 29 °C and 65% humidity, four rules fire. Their clipped outputs are aggregated and the centroid gives a cooling power of about 63.5%. The output changes smoothly with the inputs, unlike a crisp thermostat that only switches ON/OFF.

Lab Tasks

1. Plot the aggregated output fuzzy set for input (29 °C, 65%) and mark the centroid on the plot.
2. Draw a **control surface**: compute the output for all temperatures 10–45 and humidities 0–100 and plot it as a 3-D surface or heat map. Comment on its smoothness.
3. Replace centroid defuzzification with **mean of maximum (MOM)** and compare outputs for the four test inputs.
4. Add a third output variable **fan speed** (Slow / Medium / Fast) with its own rule base and display both outputs.

Post-Lab Task

Re-implement the controller using the **scikit-fuzzy** library (`pip install scikit-fuzzy`) with `ctrl.Antecedent`, `ctrl.Consequent` and `ctrl.Rule`. Verify that your from-scratch outputs match the library outputs (small differences are acceptable).

Viva Questions

1. What are the four steps of Mamdani inference?
2. What is the difference between Mamdani and Sugeno fuzzy systems?
3. Why is defuzzification needed?
4. What happens if no rule fires for an input?

References

1. T. J. Ross, *Fuzzy Logic with Engineering Applications*, 4th ed., Wiley, Chapters 5 and 8.

2. scikit-fuzzy documentation: https://pythonhosted.org/scikit-fuzzy/

Summary

Students designed a complete fuzzy expert system for AC control, implementing fuzzification, rule evaluation, aggregation and centroid defuzzification in Python.

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab # 10: Supervised and Unsupervised Learning: Naive Bayes and K-Means

Objective: To explore supervised and unsupervised learning by implementing Naive Bayes for text classification and applying K-Means clustering to the Iris dataset.

Week / Lab No.	:	Week 12 / Lab 10
CLO Mapping	:	CLO2
Time Required	:	3 hrs
Programming Language	:	Python 3.x
Software Required	:	Anaconda (Jupyter Notebook) / Google Colab
Libraries	:	scikit-learn, numpy, matplotlib

Learning Outcomes

- Distinguish between supervised, unsupervised and reinforcement learning.
- Apply Bayes' theorem to text classification using a bag-of-words model.
- Cluster unlabelled data with K-Means and choose k using the elbow method.
- Evaluate models with accuracy, precision, recall, F1 and Adjusted Rand Index.

Part A: Naive Bayes Text Classification (Supervised)

10.1 Types of Machine Learning

- **Supervised learning:** learn a mapping from inputs to known labels (classification, regression).
- **Unsupervised learning:** discover structure in unlabelled data (clustering, dimensionality reduction).
- **Reinforcement learning:** learn actions from rewards and penalties.

10.2 Naive Bayes Classifier

Using Bayes' theorem, the probability of class *c* given document *d* = (w1, w2, …, wn) is:

$$P(c \mid d) \propto P(c) \cdot \Pi\, P(wi \mid c)$$

The "naive" assumption is that words are **conditionally independent** given the class. The prior P(c) is the fraction of training documents in class *c*. For Multinomial Naive Bayes, P(w | c) = (count(w, c) + α) / (total words in c + α|V|), where α = 1 is **Laplace smoothing**, which prevents zero probabilities for unseen words.

Example Code

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

messages = [
    ("Congratulations! You have won a free iPhone, click here to claim", "spam"),
    ("WIN cash prize now!!! Limited offer, reply YES", "spam"),
    ("Get cheap loans instantly, no documents required", "spam"),
    ("You have been selected for a free holiday, call now", "spam"),
    ("Claim your reward points before they expire, click the link", "spam"),
    ("Exclusive deal: 70% discount on all products today only", "spam"),
    ("Urgent! Your account is blocked, verify your password now", "spam"),
    ("Free entry in weekly lucky draw, text WIN to 8080", "spam"),
    ("Earn money from home, guaranteed income, join free", "spam"),
    ("Lowest price medicines, order now and get free delivery", "spam"),
```

```
    ("Are we still meeting for the AI lab tomorrow?", "ham"),
    ("Please submit your lab report before Friday", "ham"),
    ("Can you share the notes of today's lecture?", "ham"),
    ("I will call you after the class ends", "ham"),
    ("The midterm exam schedule has been uploaded on LMS", "ham"),
    ("Let's have lunch at the cafeteria at 1 pm", "ham"),
    ("Mom asked you to bring groceries on the way home", "ham"),
    ("Our project meeting is moved to Monday morning", "ham"),
    ("Thanks for helping me with the Python assignment", "ham"),
    ("Is the 8-puzzle code working on your laptop?", "ham"),
]
texts, labels = zip(*messages)

X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.3, random_state=0, stratify=labels)

vectorizer = CountVectorizer(lowercase=True, stop_words='english')   # bag of words
Xtr = vectorizer.fit_transform(X_train)
Xte = vectorizer.transform(X_test)

model = MultinomialNB(alpha=1.0)        # alpha = Laplace smoothing
model.fit(Xtr, y_train)
pred = model.predict(Xte)

print("Vocabulary size:", len(vectorizer.vocabulary_))
print("Test accuracy  :", accuracy_score(y_test, pred))
print(classification_report(y_test, pred, zero_division=0))

new = ["Click here to win a free laptop now",
       "Can we discuss the open ended lab project today?"]
for msg, p in zip(new, model.predict(vectorizer.transform(new))):
    print(f"{p.upper():5s} <- {msg}")
```

Output

```
Vocabulary size: 67
Test accuracy  : 0.8333333333333334
              precision    recall  f1-score   support

         ham       0.75      1.00      0.86         3
        spam       1.00      0.67      0.80         3

    accuracy                           0.83         6
   macro avg       0.88      0.83      0.83         6
weighted avg       0.88      0.83      0.83         6

SPAM  <- Click here to win a free laptop now
HAM   <- Can we discuss the open ended lab project today?
```

Note: the dataset has only 20 messages, so the accuracy changes with the random split. In the lab tasks you will use a real dataset with thousands of messages.

Part B: K-Means Clustering (Unsupervised)

10.3 K-Means Algorithm

Pseudocode

```
1. Choose k and initialise k centroids randomly
```

```
2. repeat
3.     Assign each point to the nearest centroid (Euclidean distance)
4.     Recompute each centroid as the mean of its assigned points
5. until assignments no longer change
   Objective: minimise inertia = sum of squared distances to the centroids
```

The **elbow method** plots inertia against k; the "elbow" where the decrease slows suggests a good k. The **Iris dataset** contains 150 flowers with 4 features (sepal length/width, petal length/width) from 3 species. We ignore the labels during clustering and use them only for evaluation with the **Adjusted Rand Index (ARI)** (1 = perfect agreement, 0 = random).

Example Code

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import adjusted_rand_score
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

iris = load_iris()
X = StandardScaler().fit_transform(iris.data)     # scale features

# Elbow method: inertia for k = 1..8
inertias = [KMeans(n_clusters=k, n_init=10, random_state=0).fit(X).inertia_
            for k in range(1, 9)]
print("Inertia for k=1..8:", [round(i, 1) for i in inertias])

km = KMeans(n_clusters=3, n_init=10, random_state=0)
clusters = km.fit_predict(X)
print("Cluster sizes      :", np.bincount(clusters))
print("Adjusted Rand Index:", round(adjusted_rand_score(iris.target, clusters), 3))

fig, ax = plt.subplots(1, 2, figsize=(9, 3.3))
NAVY = '#0E367B'                        # Iqra University blue
ax[0].plot(range(1, 9), inertias, 'o-', color=NAVY); ax[0].set_title('Elbow method')
ax[0].set_xlabel('k'); ax[0].set_ylabel('Inertia')
for k, m in zip(range(3), ['o', 's', '^']):        # one marker shape per cluster
    pts = X[clusters == k]
    ax[1].scatter(pts[:, 2], pts[:, 3], marker=m, s=18, facecolors='none',
                  edgecolors=NAVY, label=f'Cluster {k}')
ax[1].scatter(km.cluster_centers_[:, 2], km.cluster_centers_[:, 3],
              c=NAVY, marker='X', s=140, label='Centroid')
ax[1].legend(fontsize=7)
ax[1].set_title('K-Means clusters (petal features)')
ax[1].set_xlabel('petal length (scaled)'); ax[1].set_ylabel('petal width (scaled)')
plt.tight_layout(); plt.savefig('lab10_kmeans.png', dpi=130)
```

Output

```
Inertia for k=1..8: [600.0, 222.4, 139.8, 114.1, 90.8, 80.0, 71.6, 62.9]
Cluster sizes      : [53 50 47]
Adjusted Rand Index: 0.62
```

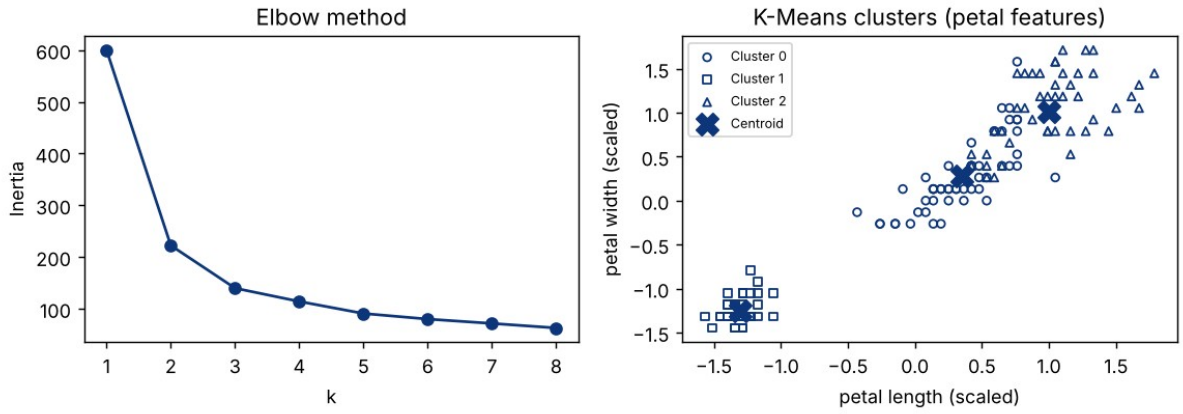


Figure 10.1: Elbow curve and K-Means clusters of the Iris dataset

Lab Tasks

1. Download the **SMS Spam Collection** dataset (UCI Machine Learning Repository / Kaggle) and train Multinomial Naive Bayes on it with an 80/20 split. Report accuracy, precision, recall, F1 and the confusion matrix.
2. Replace `CountVectorizer` with `TfidfVectorizer`. Does performance improve? Display the 10 words with the highest P(word | spam).
3. Implement **K-Means from scratch** using NumPy (no scikit-learn) and compare its cluster assignments with scikit-learn's on the Iris dataset.
4. Run K-Means on Iris for k = 2, 3 and 4 and compute the **silhouette score** for each. Which k is best and why?

Post-Lab Task

Collect 30 short Roman-Urdu or English product reviews (positive/negative), label them, and build a Naive Bayes sentiment classifier. Report its performance and discuss the challenges of Roman-Urdu text.

Viva Questions

1. Why is Naive Bayes called "naive"?
2. What is Laplace smoothing and why is it needed?
3. How does K-Means decide which cluster a point belongs to?
4. Why is feature scaling important for K-Means?

References

1. scikit-learn User Guide: Naive Bayes and Clustering, https://scikit-learn.org/stable/user_guide.html
2. T. Mitchell, *Machine Learning*, McGraw-Hill, Chapter 6.

Summary

Students trained a Naive Bayes text classifier (supervised learning) and clustered the Iris dataset with K-Means (unsupervised learning), evaluating both with suitable metrics.

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab # 11: Artificial Neural Networks: The Perceptron

Objective: To build a foundational understanding of Artificial Neural Networks by creating a simple perceptron for binary classification and training it on a linearly separable dataset.

Week / Lab No.	:	Week 13 / Lab 11
CLO Mapping	:	CLO2
Time Required	:	3 hrs
Programming Language	:	Python 3.x
Software Required	:	Anaconda (Jupyter Notebook) / Google Colab
Libraries	:	numpy, matplotlib

Learning Outcomes

- Relate the biological neuron to the artificial neuron.
- Implement the perceptron learning rule from scratch.
- Explain linear separability and the limitation of a single perceptron (XOR problem).

Theory

11.1 From Biological to Artificial Neurons

A biological neuron receives signals through dendrites, sums them in the cell body and fires an output through the axon when a threshold is exceeded. The artificial neuron (McCulloch–Pitts, 1943) imitates this: inputs are multiplied by **weights**, summed with a **bias**, and passed through an **activation function**.

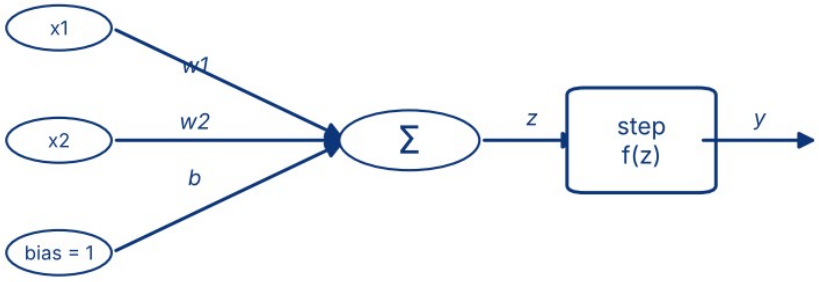


Figure 11.1: A perceptron with two inputs, a bias, weighted sum and step activation

11.2 The Perceptron (Rosenblatt, 1958)

$$z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b, \quad y = \text{step}(z) = 1 \text{ if } z \geq 0, \text{ else } 0$$

Perceptron learning rule: for each training example with target t and prediction y:

$$w_i \leftarrow w_i + \eta\,(t - y)\,x_i \quad \text{and} \quad b \leftarrow b + \eta\,(t - y)$$

where η is the **learning rate**. The **perceptron convergence theorem** guarantees that if the data are **linearly separable** (a straight line/hyperplane can separate the two classes), the rule finds a separating boundary in a finite number of updates. AND and OR are linearly separable; **XOR is not**, so a single perceptron cannot learn it.

Implementation

Example Code

```
import numpy as np
```

```
class Perceptron:
    def __init__(self, n_inputs, lr=0.1):
        self.w = np.zeros(n_inputs)
        self.b = 0.0
        self.lr = lr

    def activation(self, z):
        return 1 if z >= 0 else 0              # step function

    def predict(self, x):
        return self.activation(np.dot(self.w, x) + self.b)

    def train(self, X, y, epochs=20):
        for epoch in range(1, epochs + 1):
            errors = 0
            for xi, target in zip(X, y):
                error = target - self.predict(xi)
                if error != 0:
                    # Perceptron learning rule
                    self.w += self.lr * error * xi
                    self.b += self.lr * error
                    errors += 1
            print(f"Epoch {epoch}: misclassified = {errors}, w = {self.w}, b =
{self.b:.1f}")
            if errors == 0:
                print("Converged!")
                break

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_and = np.array([0, 0, 0, 1])

p = Perceptron(2)
p.train(X, y_and)
print("\nAND gate predictions:")
for xi in X:
    print(f"  {xi} -> {p.predict(xi)}")

# XOR is NOT linearly separable: the perceptron never converges
print("\nTraining on XOR (10 epochs):")
q = Perceptron(2)
y_xor = np.array([0, 1, 1, 0])
for epoch in range(10):
    errs = 0
    for xi, t in zip(X, y_xor):
        e = t - q.predict(xi)
        if e: q.w += q.lr * e * xi; q.b += q.lr * e; errs += 1
print("Misclassified in last epoch:", errs, "-> XOR is not linearly separable")
```

Output

```
Epoch 1: misclassified = 2, w = [0.1 0.1], b = 0.0
Epoch 2: misclassified = 3, w = [0.2 0.1], b = -0.1
Epoch 3: misclassified = 3, w = [0.2 0.1], b = -0.2
Epoch 4: misclassified = 0, w = [0.2 0.1], b = -0.2
Converged!

AND gate predictions:
  [0 0] -> 0
```

```
  [0 1] -> 0
  [1 0] -> 0
  [1 1] -> 1

Training on XOR (10 epochs):
Misclassified in last epoch: 4 -> XOR is not linearly separable
```

Observation: The perceptron converges in 4 epochs for AND. The learned boundary is 0.2x1 + 0.1x2 − 0.2 = 0. For XOR it keeps misclassifying, because no single straight line separates the classes.

Lab Tasks

1. Train the perceptron on the **OR** and **NAND** gates. Print the learned weights and bias for each.
2. Plot the four input points of the AND gate and the learned decision boundary line using Matplotlib.
3. Generate a **linearly separable dataset** of 100 points using `sklearn.datasets.make_blobs(centers=2)`. Train the perceptron, plot the boundary and report the training accuracy.
4. Study the effect of the learning rate: train on the dataset of Task 3 with η = 0.001, 0.01, 0.1 and 1. Plot the number of misclassifications per epoch for each.

Post-Lab Task

Train your perceptron on two classes of the Iris dataset (Setosa vs Versicolor) using petal length and petal width. Split 70/30, report test accuracy and plot the decision boundary. Then try Versicolor vs Virginica and explain the result.

Viva Questions

1. What is the role of the bias term?
2. What does linearly separable mean?
3. Why can a single perceptron not learn XOR?
4. What happens if the learning rate is too large?

References

1. S. Haykin, *Neural Networks and Learning Machines*, 3rd ed., Pearson, Chapter 1.
2. F. Rosenblatt, “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain,” *Psychological Review*, 1958.

Summary

Students implemented the perceptron learning algorithm from scratch, trained it on linearly separable data and observed its limitation on the XOR problem, which motivates multi-layer networks in the next lab.

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

Lab # 12: Multi-Layer Perceptron and Backpropagation

Objective: To understand multi-layer perceptrons and the backpropagation algorithm, implement an MLP from scratch, and train an MLP on the MNIST handwritten digit dataset using PyTorch (or TensorFlow).

Week / Lab No.	:	Week 14 / Lab 12
CLO Mapping	:	CLO2
Time Required	:	3 hrs
Programming Language	:	Python 3.x
Software Required	:	Anaconda / Google Colab (GPU optional)
Libraries	:	numpy, torch, torchvision

Learning Outcomes

- Describe the architecture of a multi-layer perceptron and common activation functions.
- Derive and implement forward and backward passes (backpropagation) for a small network.
- Train and evaluate an MLP for digit classification using a deep learning framework.

Theory

12.1 Multi-Layer Perceptron (MLP)

An MLP has an **input layer**, one or more **hidden layers** and an **output layer**. Each neuron in a layer is connected to every neuron in the next layer. Hidden layers with **non-linear activation functions** allow the network to learn non-linear decision boundaries such as XOR.

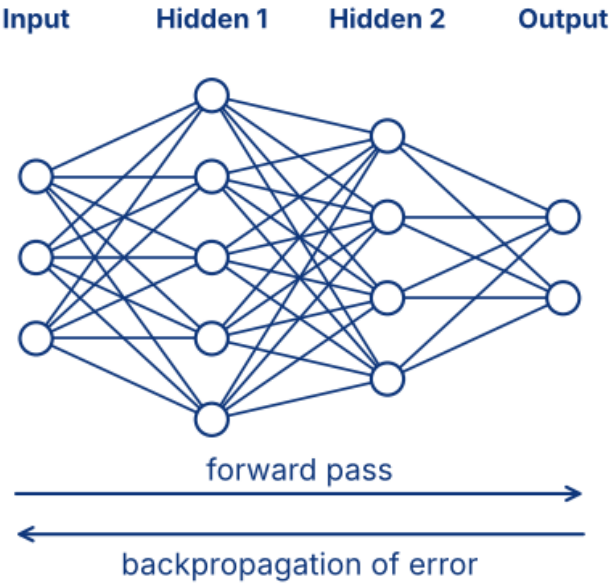


Figure 12.1: An MLP with two hidden layers; errors are propagated backwards

Activation	Formula	Typical use
Sigmoid	σ(z) = 1 / (1 + e^(−z)), σ′ = σ(1 − σ)	Binary output, small networks
ReLU	max(0, z)	Hidden layers of deep networks
Softmax	e^(zi) / Σ e^(zj)	Multi-class output layer

12.2 Backpropagation

Training minimises a **loss function** L (e.g., mean squared error or cross-entropy) using **gradient descent**. Backpropagation applies the **chain rule** to compute the gradient of the loss with respect to every weight efficiently, layer by layer from the output back to the input:

1. **Forward pass:** compute the activations of each layer and the loss.
2. **Output error:** $\delta_out = (\hat{y} - y) \odot f'(z_out)$
3. **Hidden error:** $\delta_h = (\delta_out\ W2^T) \odot f'(z_h)$
4. **Update:** $W \leftarrow W - \eta \cdot (input^T\ \delta)$ for each layer, and similarly for biases.

Part A: MLP from Scratch (XOR)

Example Code

```
import numpy as np

np.random.seed(1)
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])               # XOR

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Network: 2 inputs -> 4 hidden neurons -> 1 output
W1 = np.random.randn(2, 4); b1 = np.zeros((1, 4))
W2 = np.random.randn(4, 1); b2 = np.zeros((1, 1))
lr = 0.5

for epoch in range(1, 10001):
    # ----- Forward pass -----
    h = sigmoid(X @ W1 + b1)
    out = sigmoid(h @ W2 + b2)
    loss = np.mean((y - out) ** 2)
    # ----- Backward pass (chain rule) -----
    d_out = (out - y) * out * (1 - out)          # dL/dz2
    d_h = (d_out @ W2.T) * h * (1 - h)           # dL/dz1
    # ----- Gradient descent update -----
    W2 -= lr * h.T @ d_out;  b2 -= lr * d_out.sum(axis=0, keepdims=True)
    W1 -= lr * X.T @ d_h;    b1 -= lr * d_h.sum(axis=0, keepdims=True)
    if epoch % 2000 == 0:
        print(f"Epoch {epoch:5d}  MSE loss = {loss:.5f}")

print("\nXOR predictions after training:")
for xi, o in zip(X, out):
    print(f"  {xi} -> {o[0]:.3f} -> class {int(o[0] > 0.5)}")
```

Output

```
Epoch  2000  MSE loss = 0.00230
Epoch  4000  MSE loss = 0.00081
Epoch  6000  MSE loss = 0.00047
Epoch  8000  MSE loss = 0.00033
Epoch 10000  MSE loss = 0.00025

XOR predictions after training:
  [0 0] -> 0.008 -> class 0
  [0 1] -> 0.986 -> class 1
  [1 0] -> 0.984 -> class 1
  [1 1] -> 0.021 -> class 0
```

Observation: With one hidden layer of 4 sigmoid neurons, the network learns XOR, which a single perceptron could not learn in Lab 11.

Part B: MLP on MNIST using PyTorch

MNIST contains 70,000 grayscale images (28 × 28 pixels) of handwritten digits 0–9 (60,000 for training, 10,000 for testing). The images are flattened into 784-dimensional vectors and fed to an MLP with two hidden ReLU layers. The dataset is downloaded automatically the first time the code runs.

Example Code

```
import torch
import torch.nn as nn
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

transform = transforms.Compose([transforms.ToTensor(),
                                transforms.Normalize((0.1307,), (0.3081,))])
train_set = datasets.MNIST('data', train=True, download=True, transform=transform)
test_set  = datasets.MNIST('data', train=False, download=True, transform=transform)
train_loader = DataLoader(train_set, batch_size=64, shuffle=True)
test_loader  = DataLoader(test_set, batch_size=1000)

class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(),                 # 28x28 image -> 784 vector
            nn.Linear(784, 256), nn.ReLU(),
            nn.Linear(256, 128), nn.ReLU(),
            nn.Linear(128, 10))           # 10 digit classes
    def forward(self, x):
        return self.net(x)

model = MLP()
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(1, 6):
    model.train(); total = 0
    for images, labels in train_loader:
        optimizer.zero_grad()
        loss = loss_fn(model(images), labels)   # forward pass
        loss.backward()                         # backpropagation
        optimizer.step()                        # update weights
        total += loss.item()
    model.eval(); correct = 0
    with torch.no_grad():
        for images, labels in test_loader:
            correct += (model(images).argmax(1) == labels).sum().item()
    print(f"Epoch {epoch}: train loss = {total/len(train_loader):.4f}, "
          f"test accuracy = {100*correct/len(test_set):.2f}%")
```

Expected result: the training loss decreases every epoch and the test accuracy typically reaches about **97–98%** after 5 epochs. Exact values differ from run to run because of random weight initialisation and data shuffling.

TensorFlow / Keras equivalent

Example Code (Keras)

```
import tensorflow as tf
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(256, activation='relu'),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')])
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
model.fit(x_train, y_train, epochs=5, batch_size=64, validation_split=0.1)
print(model.evaluate(x_test, y_test))
```

Lab Tasks

1. In Part A, replace the sigmoid hidden activation with **tanh** and compare the number of epochs needed to reach a loss below 0.01.
2. In Part B, plot the training loss and test accuracy against epochs using Matplotlib.
3. Experiment with the architecture: (a) one hidden layer of 64 neurons, (b) two layers of 256 and 128, (c) three layers of 512, 256 and 128. Tabulate the number of parameters and test accuracy.
4. Add **Dropout(0.2)** after each hidden layer and compare overfitting (difference between training and test accuracy).
5. Display 10 misclassified test images with their true and predicted labels.

Post-Lab Task

Train the same MLP on **Fashion-MNIST** (`datasets.FashionMNIST`). Compare its accuracy with MNIST and explain why Fashion-MNIST is harder. Optionally, replace the MLP with a small CNN and compare.

Viva Questions

1. Why do hidden layers need non-linear activation functions?
2. Explain the chain rule as used in backpropagation.
3. What is the difference between an epoch, a batch and an iteration?
4. Why is cross-entropy preferred over MSE for classification?

References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, Chapter 6.
2. PyTorch tutorials: https://pytorch.org/tutorials/
3. TensorFlow tutorials: https://www.tensorflow.org/tutorials

Summary

Students implemented backpropagation from scratch to solve XOR and trained a multi-layer perceptron on MNIST with a modern deep learning framework, completing the progression from search to machine learning.

Lab Evaluation (to be filled by the instructor)

R1	R2	R3	R4	R5	Total (15)

Assessed using Lab Rubrics (Group 1). Each criterion is marked 0 – 3.

Date: _____ Instructor Signature: _____

APPENDIX A: LAB EXAM GUIDELINES

Midterm Lab Assessment (Week 8)

- Covers Labs 1–6: agents, problem formulation, BFS, DFS, UCS, DLS, IDDFS, Greedy, A*, hill climbing and genetic algorithm.
- Duration: 2 hours. Open-book only for this lab manual; internet and AI assistants are not allowed.
- Typical paper: one problem formulation question and one or two implementation questions, followed by a short viva.

Final Lab Examination (Week 17)

- Covers all twelve labs.
- Duration: 3 hours, including viva.
- Students must write the complete program, run it in front of the examiner and explain it.

Marking Scheme for Lab Exams

Component	Marks (%)
Correct problem formulation / approach	20%
Working code with correct output	50%
Code quality (structure, comments, naming)	10%
Viva	20%

APPENDIX B: LAB REPORT TEMPLATE

Use the following structure for post-lab tasks and the Open-Ended Lab report. Use Times New Roman 12 pt, 1.15 line spacing, and include screenshots of code output.

Section	Contents
Title page	Course code and name, lab/project title, student name(s) and registration number(s), date, instructor name
1. Objective	What the lab/project aims to achieve
2. Problem formulation	States, actions, transition model, goal test, path cost / data and features
3. Methodology	Algorithms used with pseudocode or flowchart; heuristics and parameters
4. Implementation	Key code fragments with explanation (full code in appendix or repository)
5. Results	Outputs, tables and charts comparing algorithms or models
6. Discussion	Interpretation of results, limitations and possible improvements
7. Conclusion	Main findings in 4–5 sentences
References	IEEE style citations of books, papers and websites used

Academic integrity: You may use documentation, textbooks and the references in this manual. If you use any AI assistant, you must state where and how it was used; all submitted code must be your own and you must be able to explain it.